第4章 操作系统安全

本章学习要点：

- 了解安全操作系统的安全策略与模型；
- 了解安全操作系统的访问控制机制；
- 了解安全操作系统的评测方法与准则。

操作系统是整个计算机系统的基础，它管理着计算机资源、控制着整个系统的运行，直接和硬件打交道，并为用户提供接口。无论是数据库系统、应用软件还是网络环境，它们都是建立在操作系统之上的，通过操作系统来完成对信息的访问和处理。因此，可以认为操作系统安全是整个信息安全的必要条件，这就使得操作系统经常是被攻击的目标。

WannaCry(又称 Wanna Decryptor)，是一种“蠕虫式”的勒索病毒软件，大小为 3.3MB，利用永恒之蓝(EternalBlue)进行传播。永恒之蓝是 2017 年 5 月全球范围内暴发的基于 Windows 网络共享协议进行攻击传播的蠕虫恶意代码。不法分子通过改造之前泄露的 NSA 黑客武器库中“永恒之蓝”攻击程序发起了此次网络攻击事件。

2017 年，WananCry 席卷全球，超过 100 个国家和地区因感染 WananCry 损失惨重，堪称一场科技恐怖袭击。WananCry 会扫描计算机上的 TCP 445 端口(Server Message Block/SMB)，以类似于蠕虫病毒的方式传播，攻击主机并加密主机上存储的文件，然后要求用户以比特币的形式支付赎金，勒索金额为 300～600 美元。在此次事件中，多个国家的重要信息网络受到袭击，我国的大量企业和机构内网，包括教育、企业、医疗、电力、能源、银行、交通等多个行业均受到不同程度的影响，我国的校园网络更是成为重灾区，多所高校出现病毒感染，学生的毕业论文等重要资料被病毒加密，只有支付赎金才能恢复。之后，360 安全中心发布公告，此次勒索事件是不法分子利用微软操作系统中编号为 MS17-010 的一个漏洞所致。微软公司曾经发布了对应安全补丁，遗憾的是，许多用户并没有及时更新，最终导致了这次“史无前例级别”的网络勒索事件，其中的教训令人警醒。

WananCry 感染原理：WananCry 感染的过程可分为两个阶段。

(1) 感染阶段：病毒母体 mssecsvc.exe 运行，扫描随机 IP 的计算机进行感染，在感染后释放勒索程序 tasksche.exe。

病毒在网络上设置了开关，整个程序会先试图连接 szUrl 这个域名，连接成功就关闭线程，终止感染。szUrl 是个未经注册的地址：

```
http://www.iuqerfsodp9ifjaposdfjhgosurijfaewrwergwea.com
```

这个域名显然是无效的。病毒被“启动”之后，会根据检测传给它的参数数量，安装 mssecsvc.exe，执行蠕虫函数。

蠕虫函数有三个主要作用：初始化网络、生成密码学相关的 API 和复制蠕虫的 payload

动态链接库(DLL)。payload 动态链接库的作用是把蠕虫病毒的二进制机器码复制到 C:\WINDOWS\mssecsvc.exe 并执行。初始化网络部分,蠕虫函数会生成两个线程,第一个线程用来扫描局域网内的主机,第二个线程用来扫描互联网里的主机。第一个线程通过 GetAdaptersInfo() 来获取局域网内的 IP 地址,然后用一个数组把这些 IP 地址存下来逐个扫描。这个线程会尝试连接 445 端口,如果连接成功就发起攻击感染。第二个线程会生成一个随机的 IP 地址,如果连接该随机 IP 地址的 445 端口成功,会对以 255.255.255.0 为掩码的整个地址段的计算机进行扫描,在这个地址段中开放 445 端口的计算机,都会被发起 MS17-010 漏洞攻击感染。

(2) 勒索阶段:勒索程序 tasksche.exe 运行,对磁盘文件进行加密,对感染 WananCry 的用户进行勒索。

勒索阶段主要是黑客利用 AES、RSA、比特币和洋葱路由等技术实现对感染 WananCry 的用户的匿名勒索。

洋葱网络是一种在计算机网络上进行匿名通信的技术。多层加密的通信数据在其传送到目的地的过程中,会通过由多个洋葱路由器组成的通信线路。在该过程中,每个洋葱路由器去掉一个加密层,并得到下一条路由信息,然后将数据继续发往下一个洋葱路由器,不断重复,直到数据到达目的地。

正是借助了比特币和洋葱路由技术,不法分子才能肆无忌惮地向感染 WananCry 的用户索取赎金。

WananCry 感染原因如下。

(1) 完全去中心化,没有特定的发行机构。

(2) 匿名性、无须交税及监管。

(3) 比特币依赖 P2P 网络,不会受发行机构的影响。

(4) 较方便、很简单就可以完成跨境交易。

EternalBlue(在微软的 MS17-010 中被修复)是在 Windows 的 SMB(server message block)服务处理 SMB v1 请求时发生的漏洞,这个漏洞导致攻击者在目标系统上可以执行任意代码。该漏洞出现的原因是 Windows SMB v1 中的内核态函数 srv!SrvOs2FeaListToNt 在处理 FEA(file extended attributes)转换时,会造成大非分页池(large non-paged kernel pool,一种内核的数据结构)上的缓冲区溢出。函数 srv!SrvOs2FeaListToNt 在将 FEA list 转换为 NTFEA(Windows NT FEA)list 前会调用 srv!SrvOs2FeaListSizeToNt 去计算转换后的 FEA lsit 的大小,然后会进行如下操作。

(1) srv!SrvOs2FeaListSizeToNt 会计算 FEA list 的大小并更新待转换的 FEA list 的大小。

(2) 由于错误地使用 Word 强制类型转换,最后计算出的待转换 FEA list 的大小比真正的 FEA list 大。

(3) 因为计算出的 FEA list 大小错误,当 FEA list 被转换为 NTFEA list 时,在大非分页池上就会出现缓冲区溢出问题。

攻击者利用缓冲区溢出漏洞就可以完成偷天换日,将原本应执行的程序代码操作转换为执行攻击代码。其详细的原理分析在 4.2.1 节给出。

4.1 操作系统安全概述

4.1.1 操作系统基础

操作系统(operating system,OS)是计算机系统中的一个系统软件,是计算机资源的直接管理者,操作系统为用户提供界面,是用户、应用程序和计算机硬件进行交互的“得力助手”。

操作系统在处理任务时,许多任务都与一些相关的安全问题有关,如图 4.1 所示,一个操作系统可以实现相关安全性的功能如下。

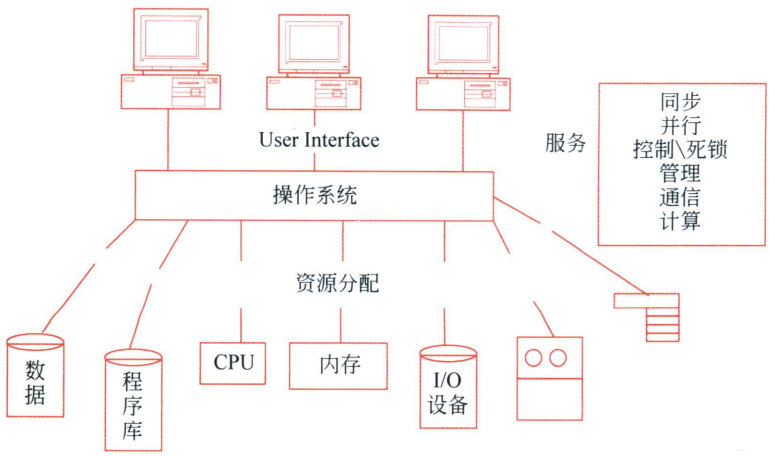

图 4.1 操作系统功能

(1) 用户鉴别。操作系统可以通过口令验证等机制对提出访问请求的用户识别并确定其身份。

(2) 内存保护。在一个系统中,所有的程序都是在受保护的内存中执行任务。这种保护可以防止外来的访问对内存造成的数据破坏,也可以控制一个用户对某一受限空间的访问权。例如,读、写、执行等不同的操作在内存中都是有不同的安全级别的。

(3) 文件及 I/O 设备访问控制。操作系统有保护 I/O 设备不受非法访问的能力。对于数据的保护通常是以查询表的形式利用访问控制矩阵来完成的。

(4) 对一般客体的资源分配与访问控制。用户在构造并发性许可并允许同步性时,要保证对客体的访问不会干扰到各个用户。通常会使用访问控制,它是由查询表实现的。

(5) 实现共享化。在系统中有些资源是分配给各用户专用的,而共享化则要求系统具有完整性和一致性。通常用带有完整性控制的查询表来实现共享。

(6) 公平服务。所有的用户都希望能最大限度地利用 CPU 得到尽可能多的服务。如不加控制,系统就会变得杂乱无章。操作系统通常使用硬件时钟控制和排序原则去保证服务公平性。

(7) 内部进程通信和同步。执行进程经常会与其他进程交换数据或保持同步性以实现资源的共享。操作系统的进程之间是相互联系的,用于交换需要处理的数据及同步信息。这些通信和同步由访问控制表实现。

(8) 操作系统数据保护。在操作系统中,重要的数据都是不能被随意更改的。很显然,如果数据没有受到保护而无法抵制非法访问,那么这个操作系统也就毫无安全性可言了。不同

的技术如加密、硬件控制和分割技术等都被用来支持操作系统的数据保护。

4.1.2 操作系统面临的威胁

操作系统需要为文件、网络及应用程序提供底层的安全保障。可以说，操作系统的安全是整个信息安全体系的基石，所以一旦操作系统存在安全缺陷和安全漏洞，会造成十分严重的后果。

操作系统面临的安全威胁主要有以下几类。

(1) 系统漏洞：漏洞问题在不同的软、硬件设备中都可能存在。不同条件下会存在不同的安全漏洞问题，如不同种类的软硬件设备、相同设备不同系统，以及相同系统不同设置等。对于操作系统而言，系统漏洞是操作系统在设计时因为逻辑上的缺陷或是代码的编写错误而导致的。漏洞的存在可能是无意的也可能是有意为之。攻击者利用系统漏洞植入木马或植入病毒攻击用户的计算机，达到控制、窃取信息或是破坏系统等目的。

(2) 特洛伊木马：特洛伊木马是一种隐藏程序，黑客将木马植入宿主计算机后，木马会在用户毫无意识的情况下运行获得宿主计算机远程访问和控制系统的权限。一般来说，特洛伊木马分为两部分，客户端和服务器端。这样的设计与攻击者利用特洛伊木马的方式有关，黑客一般会将木马隐藏在一些游戏或者不正规的网站上，用户打开后就会下载了带有恶意代码的软件，这个软件就可以视作被木马服务器部分绑定的软件。在用户运行软件的时候，木马的服务器端也就完成了安装。木马的客户端部分则被攻击者所使用进行对宿主计算机的控制。

(3) 计算机病毒：计算机病毒是一种影响计算机使用且能够自我复制的一段计算机指令或代码。一般病毒都具有以下 6 个特点。

- 寄生性：计算机病毒需要寄生在其他程序中，当程序被启动时，病毒也被启动，并开始破坏行为。
- 传染性：病毒具有自我复制的能力与变异能力，计算机病毒也一样，能够像传染病一样快速地感染目标计算机。
- 潜伏性：计算机病毒隐藏在程序中，根据预先设计的启动条件，当条件满足时暴发开来，对计算机系统进行破坏。
- 隐蔽性：计算机病毒隐藏在程序中，在未暴发时很难通过杀毒软件检查出来。
- 破坏性：计算机病毒对系统的破坏力巨大，可以导致系统程序无法运行，疯狂复制或删除文件等。
- 可触发性：计算机病毒可以根据预先设计的某个事件触发，开始对计算机系统实施感染和攻击。

(4) 隐蔽通道：隐蔽通道指系统中不受安全策略控制的、违反安全策略的信息泄露路径。在实施了强制访问控制的操作系统中，攻击者必须利用隐蔽通道，才能使植入的特洛伊木马服务端将信息传递给木马的客户端。隐蔽通道分为隐蔽存储通道和隐蔽定时通道。

- 隐蔽存储通道：如文件节点号信道，隐蔽存储通道的作用是当一个进程直接或间接写入一个存储位置，另一个进程可以直接或者间接读这个位置。
- 隐蔽定时通道：隐蔽定时通道的作用是当一个进程调节自己对系统资源(如 CPU)的使用、从而影响另外一个进程观察到的真实系统响应时间，如 CPU 调度信道。

1. 安全操作系统

操作系统安全一般指在操作系统的基本功能上为了保证计算机资源使用的保密性、完整

性和可用性，增加了相应的安全机制和措施。安全操作系统的概念与操作系统安全不同。安全操作系统是针对安全性开发增强的操作系统，通常与相应的安全级对应。例如，根据TCSEC标准，达到安全等级B1以上的操作系统，可以被定义为安全操作系统。

对于任何一个操作系统，可以说它们都具有一定的安全性，却不能说它们都是安全操作系统。但二者又是统一和密不可分的，因为它们都在讨论系统的安全性。

2. 相关定义及术语

可信计算基（Trusted Computing Base，TCB）：计算机系统内保护装置的总体，包括硬件、固件、软件和负责执行安全策略的组合体。它建立了一个基本的保护环境并提供一个可信计算系统所要求的附加用户服务。

内核（kernel）：操作系统中为众多应用程序提供对计算机硬件的安全访问程序。

安全策略（security policy）：在TCB中对所有与安全相关的活动和资源进行分配、保护和管理的一组规则。

安全模型（security model）：用形式化的方法来描述如何实现系统的机密性、完整性和可用性的安全要求。

主体（subject）：引起信息在客体中流动的实体。

客体（object）：系统中接受主体活动的承担者。

授权（authorization）：授予用户、程序或进程访问权的动作。

访问控制（access control）：限制任何未授权的资源访问。

自主访问控制（Discretionary Access Control，DAC）：用来决定一个用户是否有权限访问此客体的一种访问约束机制，该客体的所有者可以按照自己的意愿指定系统中的其他用户对此客体的访问权。

敏感标记（sensitivity label）：用以表示客体安全级别并描述客体数据敏感性的一组信息，在可信计算基中把敏感标记作为强制访问控制决策的依据。

强制访问控制（Mandatory Access Control，MAC）：用于将系统中的信息分密级和类进行管理，以保证每个用户只能够访问那些被标明可以由他访问的信息的一种访问约束机制。

角色（role）：系统中一类访问权限的集合。

审计（audit）：审核系统中有关安全的活动。

最小权原则（least privileges principle）：为了保证发生错误或事故后造成的损失最小化，系统会限定每个主体所必需的最小特权。

隐蔽信道（covert channel）：允许进程以危害系统安全策略的方式传输信息的通信信道。

客体重用（object reuse）：对曾经包含一个或几个客体的存贮介质（如页框、盘扇面、磁带）重新分配和重用。为了安全地进行重分配、重用，要求介质不得包含重分配前的残留数据。

可信通路（trusted path）：终端人员能借以直接同可信计算基通信的一种机制。该机制只能由有关终端操作人员或可信计算基启动，并且不能被不可信软件模仿。

多级安全（Multilevel Secure，MLS）：一类包含不同等级敏感信息的系统，它既可供具有不同安全许可的用户同时进行合法访问，又能阻止用户去访问其未被授权的信息。

安全操作系统（secure operating system）：能对所管理的数据与资源提供适当的保护级，有效地控制硬件与软件功能的操作系统。就安全操作系统的形成方式而言，一种是从系统开始设计时就充分考虑到系统的安全性的安全设计方式；另一种是基于一个通用的操作系统，专门进行安全性改进或增强的安全增强方式。安全操作系统在开发完成后，在正式投入使用之

前一般都要求通过相应的安全性评测。

多级安全操作系统(multilevel secure operating system)：实现了多级安全策略的安全操作系统，如符合美国 TCSEC B1 级以上的安全操作系统。

4.2 操作系统内核安全

内核是操作系统实现最底层功能的部分，是操作系统的核心组件。在一个标准的操作系统设计中，如同步、进程间通信、消息传递和中断处理等所有的服务操作都是以系统调用的方式通过内核实现的。在现代操作系统中，内核负责提供一些常规化的服务，如管理虚拟内存、硬件驱动访问、输入输出处理(如键盘、鼠标或视频显示器)等。内核通常是由汇编语言或 C 语言混合编写而成的一段代码，这些代码由于底层的体系架构机制将与其他运行的程序分隔开来。

内核是整个操作系统的工作基础，具有整个操作系统的最高权限，操作系统内核的安全会影响到整个操作系统乃至上层应用的安全。所以，对于一个安全和稳定的系统来说，保证操作系统内核的安全，是保证系统安全的第一步。

4.2.1 Windows 系统内核安全

1. Rootkit

Rootkit 有很多小型程序组成的工具包，攻击者可以使用它成为计算机上的最高权限用户。换句话说，Rootkit 是能够持久、可靠并无法检测地存在于计算机之上的一组程序或代码。要注意的是，Rootkit 只是一种技术，并不一定是恶意代码。大量的合法程序会使用 Rootkit 技术进行远程管理。当然，也有部分企业将其用作窃听或作为潜行技术。

Rootkit 技术的发展与计算机的发展息息相关，随着社会的信息化发展，人们对于计算机的控制欲望逐渐上升。Rootkit 使用方便，当使用者需要持续访问时，Rootkit 发挥作用，提供远程控制。在取得信息后，可将 Rootkit 删除，系统上不会留下任何操作痕迹。

2. Windows 内核主要功能组件

内核功能组件包括：进程管理、文件访问、安全性和内存管理。

进程管理：所有的进程都由内核管理公平地共享 CPU 资源。内存中的数据结构记录了所有线程和进程。攻击者通过修改这部分代码可以隐藏进程。

文件访问：内核可以加载设备驱动来处理底层的文件系统。内核对这些文件系统提供一致的接口。通过修改内核中这部分代码，攻击者可以隐藏文件和目录。

安全性：进程之间的约束由内核负责实施，系统内核为每个进程都实施权限控制和独立的内存空间。对内核的这部分代码只需要进行少量改动就可以删除所有的安全机制。

内存管理：对于计算机而言，一个内存地址可以映射到多个物理位置，同一地址可能指向两个包含不同数据的完全不同的物理内存位置。对内核此部分进行修改，攻击者可以达到对调试器或取证软件隐藏数据的目的。

3. 案例分析

介绍了关于 Windows 内核基础后，下面以重写任意内存和堆栈缓冲区溢出两个例子，说明攻击者如何利用内核漏洞影响目标。

(1) 重写任意内存：也叫 write-what-where 漏洞，是最常见的影响 Windows 内核驱动的漏

洞。这种漏洞主要是由于失败或不正确地使用用户态验证的内核 API。write-what-where 漏洞可以直接导致一系列的缓冲区溢出、逻辑错误等。通常，面对这样一个漏洞，攻击者可以重写一个被控制的内存地址的多个字节，这些字节的内容无论被部分控制或完全控制，攻击者都可以拥有重写的所有权限。

（2）堆栈缓冲区溢出：指当计算机向缓冲区内填充数据位数超过了缓冲区本身的容量溢出的数据覆盖在合法的数据上。理想的情况是程序检查数据长度并不允许输入超过缓冲区长度的字符，但是绝大多数程序都会假设数据长度总是与所分配的存储空间相匹配，这就为缓冲区溢出埋下隐患。操作系统所使用的缓冲区，又被称为“堆栈”。在各个操作进程之间，指令会被临时存储在“堆栈”当中，“堆栈”会出现缓冲区溢出。针对该漏洞，攻击者只需要往程序的缓冲区写超出其长度的内容，从而造成缓冲区的溢出，破坏程序的堆栈，使程序转而执行其他指令，就可以达到其目的。

4. 教学实例

目标：理解堆栈缓冲区溢出的过程与原理，观察变量、堆栈数据的变化情况，掌握 Windows 操作系统的存储分配、局部内存变量、堆栈和函数调用之间的关系。进一步思考如何防范基于缓冲区溢出的攻击。以 Windows XP 操作系统和 Virtual C++ 软件工具为例。

（1）在 Virtual C++ 工具中，选择新建 win32 控制台应用程序，并将工程命名为 test。在该工程中，新建 experiment.cpp 文件，具体创建过程如图 4.2 和图 4.3 所示，准备编写一个简单的 C++ 程序，借助该程序来理解堆栈缓冲区溢出的过程与原理。

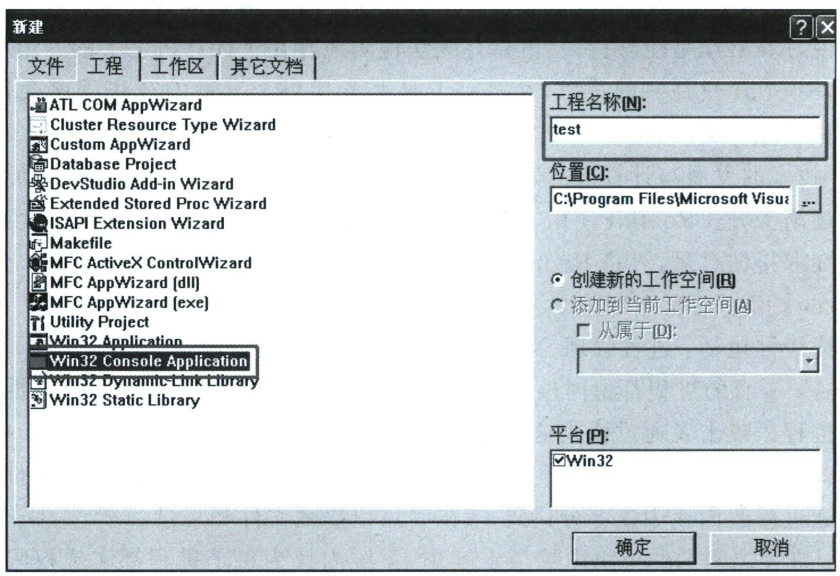

图 4.2　创建应用程序示例图

（2）在 experiment.cpp 文件中编写主函数和被调用的子函数代码，代码详情如图 4.4 和图 4.5 所示。

（3）为了观察函数调用时堆栈发生的变化，在指定行设置断点，如图 4.6 所示。

（4）按 F5 跟踪源码执行状态，程序运行结果如图 4.7 所示。

（5）按 1 键时，程序的中间变量变化情况如图 4.8 所示。由于申请了一个八块区域的数组，但却给它赋值了十一块的内容，导致了变量溢出，从而覆盖了 c 的值。

（6）按 2 键，观察局部变量可以发现子函数的返回地址被覆盖掉，如图 4.9 所示，导致无法

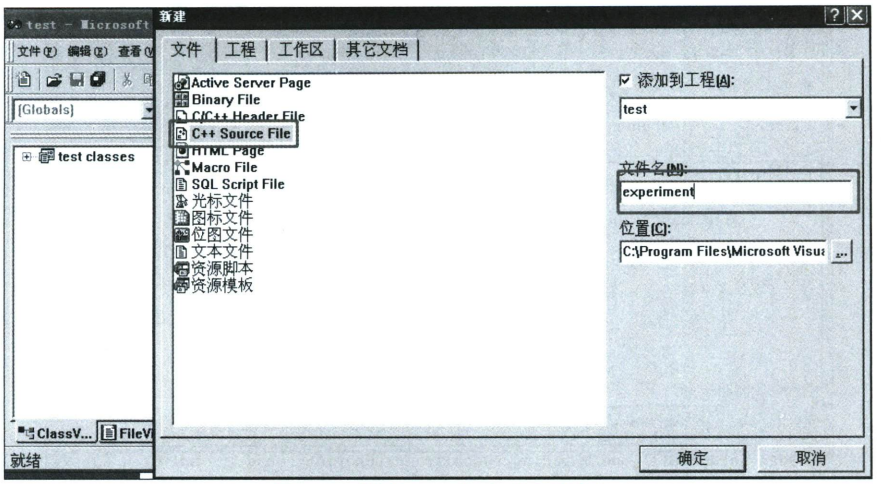

图 4.3 创建 C++ 源文件示例图

```
int main()
{
    int i=0,j=0;
    printf("按1临近变量覆盖，按2为返回值覆盖！ ")
    scanf("%d",&i);
    if(i==1)
    {
        char sz_In2[]="1234567890";
        j=fun(sz_In2,888);
    }
    else if(i==2)
    {
        char sz_In3[]="123456789abcdefghijklemnksjssjdfj";
        i=fun(sz_In3,888);
    }
    else
    {
        char sz_In1[]="123";
        j=fun(sz_In1,888);
    }
    return 0;
}
```

图 4.4 主函数代码图

```
int fun(char *szIn,int nTest)
{
    int a=1,b=2,c=0;
    char szBuf[8];
    memcpy(szBuf,szIn,strlen(szIn));
    c=a+b;
    return c;
}
```

图 4.5 子函数代码图

正常退出。

(7) 退出程序，按 F5 重新运行，此时 a、b、c 的值已经被全部改变，按 F5 继续执行代码会弹出错误提醒，如图 4.10 所示。

4.2.2 Linux 系统内核安全

Linux 内核与 Windows 内核相比，最大的区别是 Linux 内核代码是开源的，这意味着对其内核代码进行修改更加便利。通过加载内核模块，可以在运行时动态地更改 Linux。可动

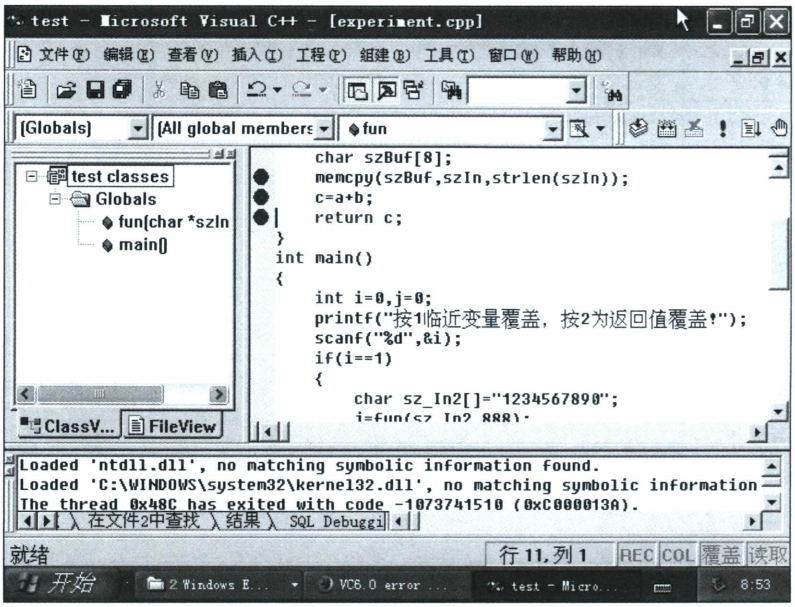

图 4.6 断点设置图

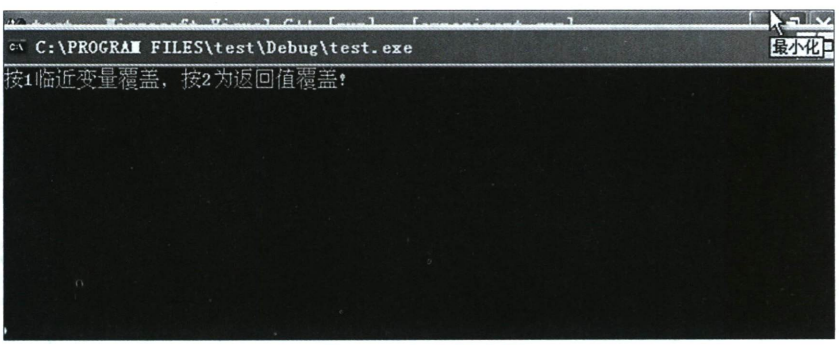

图 4.7 程序调试图

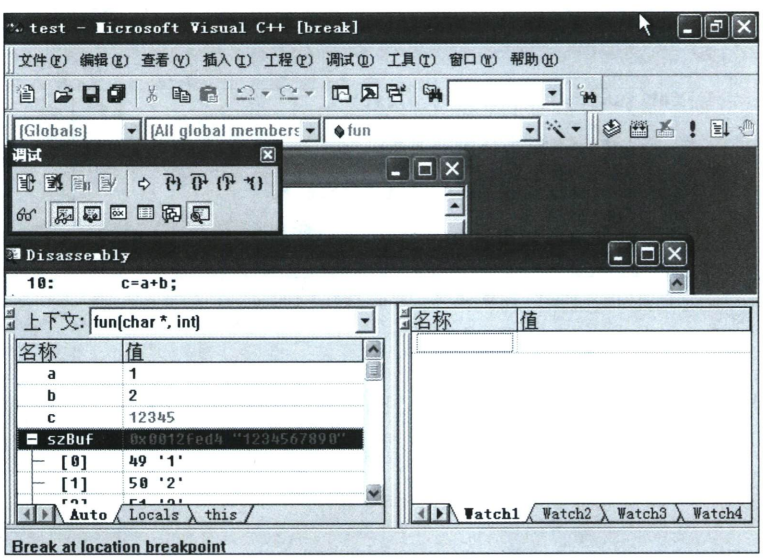

图 4.8 调试结果图 1

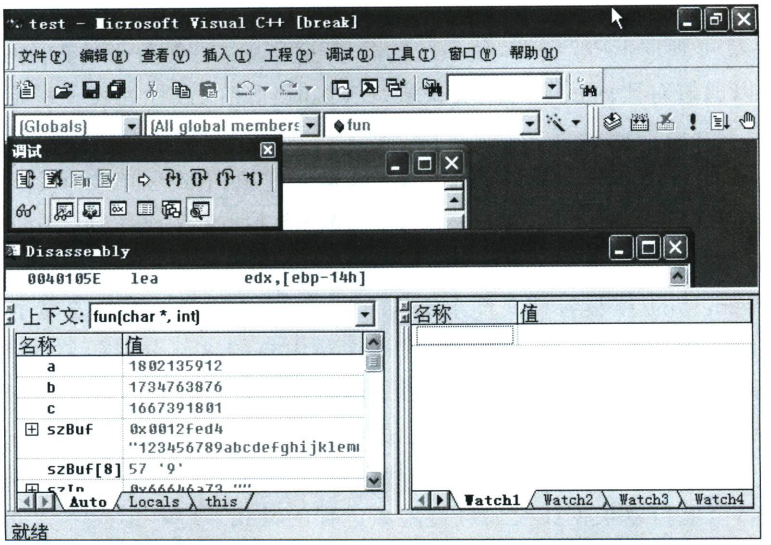

图 4.9 调试结果图 2

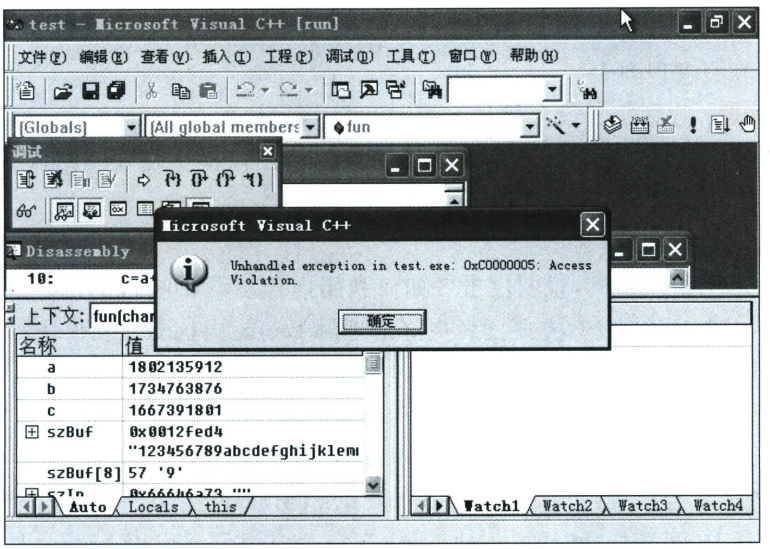

图 4.10 错误运行提示图

态更改指可以将新的功能加载到内核或者从内核去除某项功能。

但是可加载内核模块设计在带来便利的同时，也带来了新的风险。首先，这可能被恶意地利用此模块在内核中注入恶意代码，如 4.2.1 节提到的 Rootkit；其次，这会导致一定的性能损失和内存开销；最后，如果加载模块存在代码不规范等问题，可能会导致内核系统崩溃，系统宕机。

1. Linux Rootkit

用户态 Rootkit：用户态 Rootkit 可以通过加载 LD_PRELOAD 等黑客自定义的恶意库文件来实现隐藏，或者在现有进程中直接加载恶意模块来实现隐藏。比如，可以在 Apache 进程中注入恶意动态加载库来实现远程控制和隐藏的 Rootkit。

内核态 Rootkit：内核态 Rootkit 可以通过可加载内核模块将恶意代码直接加载进内核中。相比用户态 Rootkit，内核态 Rootkit 更加隐蔽，通常包含后门。

2. Linux 内核主要功能组件

进程管理：负责管理 CPU 资源，使系统的各个进程以尽可能公平的方式使用 CPU 资源。在 Linux 中，可以通过对比内核驱动模块获取系统实时创建的进程列表和系统审计工具 ps 等获取的进程列表去寻找隐藏进程。

内存管理：负责管理和分配系统内存资源，其提供虚拟内存机制，保证各个进程都享有独立的内存空间。与 Windows 不同，Linux 每个进程都有独立的内存空间，因此可以根据其占用真实内存情况寻找隐藏进程。

虚拟文件系统：负责屏蔽各种文件系统的差异，并对各种硬件驱动如显示器、键盘或磁盘进行封装，提供统一的文件接口供应用程序使用。对内核的这部分代码修改可以用新编写的函数地址替换原始文件目录地址，将原文件目录隐藏。

网络系统管理：负责管理各种网络设备，并实现各种网络协议。Rootkit 木马感染 Linux 内核后，无论如何将进程、文件等各种信息进行隐藏，其最终目的都是窃听、控制。这些都需要与远程主机通信，因此可以根据此模块进行隐藏进程的寻找。

进程间通信管理：当父进程创建子进程时，会产生父子进程通信的问题，不同进程为了完成某项任务也有通信的需要。当 Rootkit 木马感染 Linux 内核后，其进程虽被隐藏，但该隐藏进程与其他进程的通信痕迹却未必完全消除，可以通过检测进程间的通信记录，寻找和 Rootkit 有关的隐藏进程列表。

设备驱动管理：隐藏字符设备模块和网络设备等模块的工作细节，并通过独立于具体设备类型的一组接口对设备进行操作。同样地，也可以使用该模块发现网络设备与远程主机网络通信的依据，找到隐藏的进程或文件。

3. 案例分析

以 Rootkit 木马植入为例，说明攻击者如何利用内核漏洞影响目标，这里的目标是 Linux 操作系统。同时，以 NFS 为例，方便读者更好地理解 Linux 内核的安全模块。

（1）Rootkit 木马植入：黑客首先定位目标主机上的漏洞，利用漏洞和相应的提权工具进行提权，非法获得 root 权限，在提权工具成功后安装 Rootkit，之后擦除痕迹（如删除本地日志、操作历史等），随后黑客就可对被植入 Rootkit 的主机进行窃听和远程操控。

（2）NFS 的配置与检查：NFS(network file system)即网络文件系统，它允许网络中计算机之间通过 TCP/IP 网络共享资源，本地 NFS 客户端应用可以启用 Linux 内核中的 NFS 服务透明地读写位于远端 NFS 服务器上的文件，NFS 具体是由 RPC 协议实现的，RPC 主要涉及了三个小模块：一是 RPC 与用户层的接口，二是 RPC 的逻辑控制框架，三是 RPC 的通信框架。

4. 教学实例

目标：掌握 NFS 的基本概念，熟悉 NFS 服务器的配置与管理，掌握 Linux 系统之间资源共享和互访的方法。

（1）为了检查 NFS 共享功能，在 Linux 环境下打开 NFS 服务器客户端，进入终端，输入“rpm -qa|grep nfs”通过 RPM 软件包管理器检测 NFS 是否安装。编辑配置文件 vim /etc/exports 以配置 NFS 服务器，在文件中输入“/tmp *(sync,ro) 127.0.0.1(sync,rw) ”来设置要输出的共享目录客户端主机地址、输出目录的读写权限和用户映射选项等，如图 4.11 所示。

（2）输入命令 chkconfig nfs on 以设置 NFS 为开机时启动；再输入命令“chkconfig --list | grep nfs”查看是否存在 NFS 服务，如图 4.12 所示。

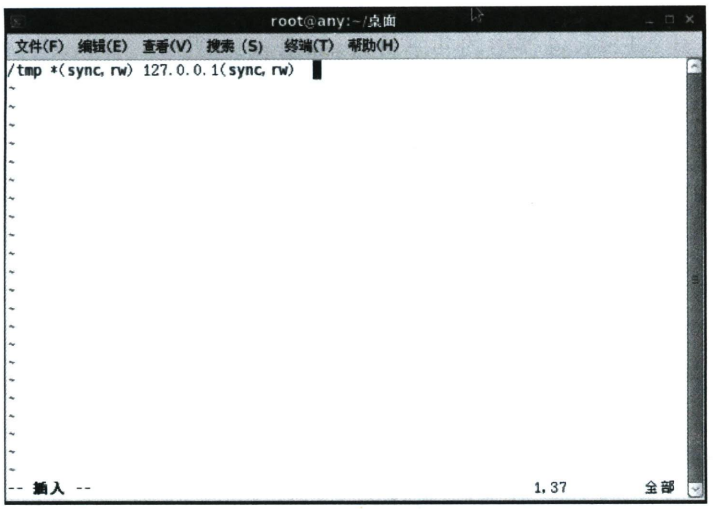

图 4.11 NFS 配置图

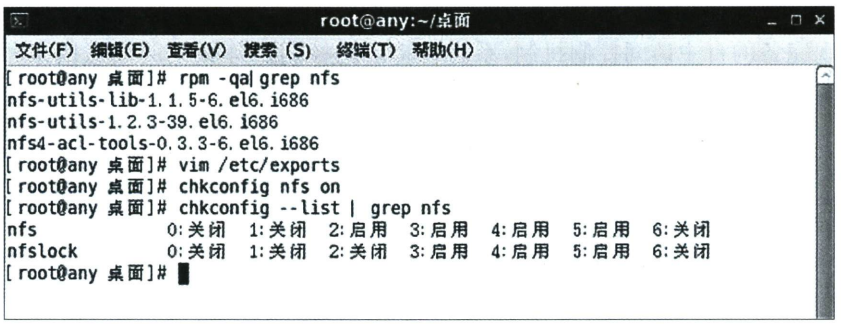

图 4.12 NFS 检测结果图

(3) 输入命令 service nfs restart 重新启动 NFS 服务,结果如图 4.13 所示。

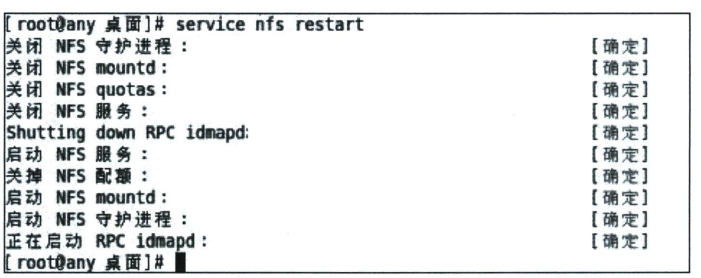

图 4.13 NFS 重启结果图

(4) 输入“showmount -e [NFS 服务器主机地址]”命令查询 NFS 服务器的共享目录状态,即显示连接到指定 NFS 服务器的所有主机,结果如图 4.14 所示。

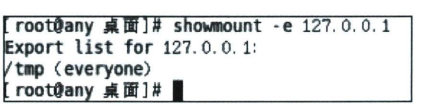

图 4.14 NFS 共享目录状态图

(5) 输入“mount [NFS 服务器主机地址]:[共享目录] [本地挂接点目录]”命令以将 NFS 服务器中共享的目录挂载到本地端的文件系统中,让不同的操作系统、机器可以共享文

件，具体如图 4.15 所示。

（6）卸载已挂载 NFS 服务器中的共享目录，如图 4.16 所示。

```
[root@any 桌面]# mount 127.0.0.1:/tmp/ /any/
[root@any 桌面]#
```

图 4.15 NFS 挂载图

```
[root@any 桌面]# umount /any/
[root@any 桌面]#
```

图 4.16 NFS 卸载图

4.3 操作系统安全机制

4.3.1 访问控制

1. 自主访问控制

自主访问控制（Discretionary Access Control，DAC）是最常用的一类访问控制机制，是用来决定一个用户是否有权访问一些特定客体的一种访问约束机制。需要自主访问控制保护的客体数量取决于系统环境，大多数的系统在自主访问控制机制中都包括对文件、目录、IPC 及设备的访问控制。

为了实现完备的自主访问控制机制，系统要将访问控制矩阵相应的信息以某种形式保存在系统中。目前在操作系统中实现的 DAC 机制是基于矩阵的行或列来表达访问控制信息的。

（1）基于行的自主访问控制机制：基于行的自主访问控制机制在每个主体上都附加一个该主体可访问的客体明细表，根据表中信息的不同又可分成以下 3 种形式。

- 能力表（capabilities list）。能力表决定用户是否可以对客体进行访问，以及进行何种模式的访问（读、写、执行），拥有相应能力的主体可以按照给定的模式访问客体。
- 前缀表（profiles）。对每个主体赋予的前缀表，包括受保护客体名和主体对它的访问权限。当主体要访问某客体时，自主访问控制机制将检查主体的前缀表是否具有它所请求的访问权。
- 口令（password）。在基于口令机制的自主访问控制机制中，每个客体都相应地有一个口令。主体在对客体进行访问前，必须向操作系统提供该客体的口令。如果正确，它就可以访问该客体。

（2）基于列的自主访问控制机制：基于列的自主访问控制机制在每个客体上都附加一个可访问它的主体明细表，它有两种形式，即保护位和访问控制表。

- 保护位（protection bits）。这种方法对所有主体、主体组及客体的拥有者指明一个访问模式集合。保护位机制不能完备地表达访问控制矩阵，一般很少使用。
- 访问控制表（Access Control List，ACL）。这是国际上流行的一种十分有效的自主访问控制模式，它在每个客体上都附加一个主体明细表，表示访问控制矩阵。表中的每一项都包括主体的身份和主体对该客体的访问权限，其一般结构如图 4.17 所示。

访问主体（subject）	对应属性（attributes）
D	Write
C	Read
B	Read/Write
A	Read

图 4.17 访问控制表

DAC在进行授权时具有很好的灵活性，适用范围广。但是，DAC在应用过程中仍存在以下几个缺陷。

- DAC允许权限进行委托，这会增加系统的风险性。
- DAC无法适应多域安全策略环境，以及在环境策略变化的情况下无法保证整体的安全性。
- DAC机制易遭到特洛伊木马攻击。
- 当主体和客体的数量较大时，会给系统带来巨大开销，因此DAC不能应用在较大规模的网络环境。

2. 强制访问控制

强制访问控制(Mandatory Access Control，MAC)可以实现比自主访问控制更为严格的访问控制策略。它的主要特点是系统对访问主体和受控对象实行强制访问控制，系统中的每个进程、每个文件、每个IPC客体(消息队列、信号量集合和共享存储区)都被赋予了相应的安全属性，这些安全属性是不能改变的，它由管理部门或由操作系统自动地按照严格的规则来设置，不像访问控制表那样可以由用户或它们的程序直接或间接地修改。

MAC设计的基本思想是：在访问控制系统中，为主体和客体分配相应的安全属性，然后系统对二者所拥有的属性关系进行判定，从而确定是否允许访问操作。在基于MAC的系统中，维护系统的管理人员会提前配置主体和客体的安全属性(或是系统自动生成的)，这种安全属性是不允许被任意修改的。在系统中，安全属性会用安全级别、类型集合等组合表示。其中类型集合是一定的元素的集体，而安全等级则表示相应的保密等级。

MAC具有强制性和严格的单向不可逆性。MAC系统规定高安全级别主体可以得到低安全级别主体的信息，而低级别主体不允许得到高级别主体的信息。也就是说，如果主体A的级别不低于客体B，那么主体A可以读取客体B中的信息和资源；如果主体A的级别不高于主体B的级别，那么主体A可以对主体B进行写入的操作；这样就保证了信息一定是流向保密级别更高的主体。MAC模型的信息始终遵循单向流通的规则，简单来说就是“不向上读，不向下写”，这也是MAC模型保证系统信息安全的基础。强制访问控制信息流向图如图4.18所示。

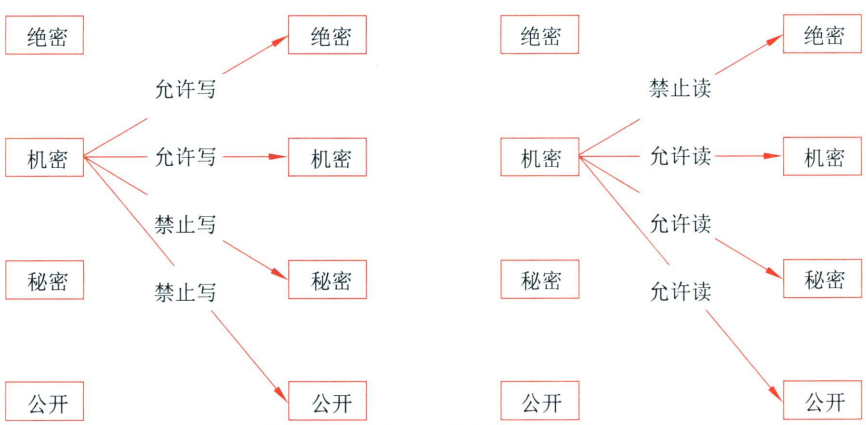

图4.18 强制访问控制信息流向图

DAC的最大特点是自主，即资源的拥有者对资源的访问策略有决策权，这在保证了授权灵活性的同时，也带来了安全隐患。MAC相较于DAC，设置了主客体的安全属性和更加强硬的控制机制，丧失了灵活性，提供了更强的安全保护。

由于 MAC 常用于将系统中的安全级别、类型集合与信息的保密级别和分类进行关联，因此广泛应用于政府部门、军队和金融等领域。

3. 基于角色的访问控制

基于角色的访问控制（Role-Based Access Control，RBAC）是传统的扩展和延伸，在 RBAC 中，权限与角色有很大联系，当用户成为某一角色之后就会得到相应的权限，给后续管理权限的工作带来了很大便利。RBAC 支持三个著名的安全原则：最小权限原则、责任分离原则和数据抽象原则。最小权限原则可以将角色根据需要设置成最小集合；责任分离原则让两个互不相容的角色来共同完成某一项任务；数据抽象原则是让角色的权限抽象化。

RBAC 认为权限授予实际上是 Who、What、How 的问题。Who、What、How 组成了该模型的访问权限三元组，也就是“谁对什么物体进行了什么样的操作”。

（1）Who：权限的拥有者或主体（如 Principal、User、Group、Role、Actor 等）。

（2）What：权限针对的对象或资源（Resource、Class）。

（3）How：具体的权限（Privilege，正向授权与负向授权）。

RBAC 的基本思想是将访问许可权分配给一定的角色，用户通过饰演不同的角色获得角色所拥有的访问许可权。RBAC 从控制主体的角度出发，根据管理中相对稳定的职权和责任来划分角色，将访问权限与角色相联系，这点与传统的 MAC 和 DAC 将权限直接授予用户的方式不同；通过给用户分配合适的角色，让用户与访问权限相联系。角色成为访问控制中访问主体和受控对象之间的一座桥梁，如图 4.19 所示。

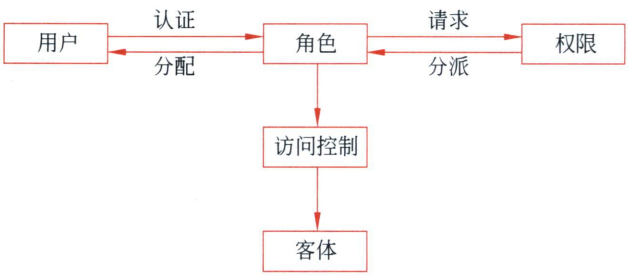

图 4.19　基于角色的访问控制模型示意图

用户即访问计算机资源的主体；角色即一种岗位，代表一种资格、权利和责任；权限即对客体的操作权力。用户分配即将用户与角色关联；权限分配即将角色与权限关联。

角色可以看作一组操作的集合，不同的角色具有不同的操作集，这些操作集由系统管理员分配给角色。在下面的实例中，假设 Tch1，Tch2，Tch3，…，Tch$i$ 是对应的教师，Stud1，Stud2，Stud3，…，Stud$j$ 是相应的学生，Mng1，Mng2，Mng3，…，Mng$k$ 是教务处管理人员，老师的权限为 TchMN＝{查询成绩、上传所教课程的成绩}；学生的权限为 StudMN＝{查询成绩、反映意见}；教务管理人员的权限为 MngMN＝{查询、修改成绩、打印成绩清单}。依据 RBAC 的策略，系统定义了各种角色，每种角色可以完成一定的职能，不同的用户根据其职能和责任被赋予相应的角色，一旦某个用户成为某角色的成员，则此用户可以完成该角色所具有的职能。

系统管理员负责授予用户各种角色的成员资格或撤销某用户具有的某个角色。例如，学校新进一名教师 Tch$x$，那么系统管理员只需将 Tch$x$ 添加到教师这一角色的成员中即可，而无须对访问控制列表做改动。同一个用户可以是多个角色的成员，即同一个用户可以扮演多种角色。比如，一个用户可以是老师，同时也可以作为进修的学生。同样，一个角色可以拥有

多个用户成员，这与现实是一致的，一个人可以在同一部门中担任多种职务，而且担任相同职务的可能不止一人。因此RBAC提供了一种描述用户和权限之间的多对多关系，角色可以划分成不同的等级，通过角色等级关系来反映一个组织的职权和责任关系，这种关系具有反身性、传递性和非对称性特点，通过继承行为形成了一个偏序关系，如MngMN＞TchMN＞StudMN。RBAC中通常定义不同的约束规则来对模型中的各种关系进行限制，最基本的约束是“相互排斥”约束和“基本限制”约束，分别规定了模型中的互斥角色和一个角色可被分配的最大用户数。RBAC中引进了角色的概念，用角色表示访问主体具有的职权和责任，灵活地表达和实现了企业的安全策略，使系统权限管理在企业的组织视图这个较高的抽象集上进行，从而简化了权限设置的管理，从这个角度看，RBAC很好地解决了企业管理信息系统中用户数量多、变动频繁的问题。

RBAC是实施面向企业安全策略的一种访问控制方式，允许组织根据用户或角色的独特需要和要求选择性地向其授予管理权限，从而应用最小特权安全原则，还具有灵活性、方便性和安全性的特点，RBAC与DAC和MAC的区别如下。

(1) DAC模型允许资源的拥有者对资源的访问策略有决策权；而在RBAC则只有系统管理员有权定义和分配角色。RBAC用户与客体无直接联系，他只有通过角色才享有该角色所对应的权限，从而访问相应的客体。因此用户不能自主地将访问权限授给别的用户，这是RBAC与DAC的根本区别所在。

(2) MAC是基于多级安全需求的，而RBAC则不是。

4.3.2 其他安全机制

1. 用户的标识与鉴别

用户是所有主体访问行为的发起源头。标识是给用户赋予的身份证明，对于操作系统而言，其给予用户的标识就是用户的账号。鉴别是系统对用户身份证明确认的过程，在操作系统中，最常见的就是口令鉴别。

标识和鉴别机制作用于用户登录的过程，以保证只有经过系统认证的用户才能对操作系统中的资源进行操作。标识和鉴别机制主要功能如下。

(1) 鉴别功能：用户在登录时，系统需要根据用户的标识检查其口令的正确性。对于鉴别信息，规定只能由用户或系统管理员才能进行修改。

(2) 标识功能：操作系统提供的标识功能应该确保用户的唯一性和独特性。用户的标识信息应由系统管理员进行管理。

(3) 对安全机制的支持：用户身份进行身份鉴别时，系统会建立一个登录进程与用户交互以得到用于身份鉴别的必要信息。这个过程需要用户提供唯一标识符给TCB(trusted computing base)，由TCB对用户进行鉴别。因此，可以看作由TCB完成的用户主体的绑定。

2. 安全审计

任何操作系统都不可能完全杜绝安全事故的发生。安全审计就是一种对历史数据进行分析、处理和追踪的安全技术，其目的是保证信息系统更加安全可靠地运行。审计系统会把可疑数据、入侵信息和敏感信息等记录下来，作为跟踪和取证使用。

安全审计一般有如下几个要求。

(1) 记录系统中发生的安全事件，并能供用户查询。

(2) 记录所有入侵行为。

（3）应该具有一定的入侵检测能力。

（4）审计系统本身必须是安全的，且能够与操作系统的其他安全机制相配合。

审计过程的实现可以分为3步。

（1）收集审计事件，产生审计记录，审计范围包括操作系统和各种应用程序。

（2）根据审计记录，进行安全事件的分析。

（3）采取处理措施。

安全审计机制主要在用户对信息进行访问、修改等操作时进行审计。审计功能一般是与其他功能隔离开的独立功能。对于操作系统而言，必须严格限制未经授权的用户访问审计数据，防止审计过程遭到非法用户的影响。

通过审计，可以达到以下两个目标。

（1）对受损的系统进行损失评估和系统恢复。

（2）详细记录与系统安全相关的行为，对这些行为进行分析，发现可能影响系统安全的不稳定因素。

3. 文件系统安全

文件系统是操作系统重要的组成部分，操作系统安全的主要问题之一就是界定哪些用户可以访问哪些资源，这些资源就包括文件资源。因此，对文件访问控制权限的判定，是操作系统的必要能力。例如：UNIX系统使用文件权限矩阵去表示哪个用户可以对文件进行何种操作；Windows操作系统则为用户提供了文件共享安全机制，当多人共用一台计算机时，为了防止不同用户打开并修改属于别人的私有文件，Windows允许用户以内容复选框的方式去保护私人数据。此外，操作系统允许系统管理员对文件系统的数据进行复制，以防止系统在运行过程中因为突发事件（如黑客入侵、停电等）导致文件系统损坏或数据丢失。

4. 内存系统安全

内存系统安全是操作系统安全的一项基本要求，也是操作系统具备的最基本的安全机制。其目的是保护用户存储数据的安全。

操作系统对于内存存取方面的安全机制有以下几方面。

（1）对于操作系统而言，一个进程运行时，操作系统会分配给其相应的内存区，内存区也被称为地址空间。在进程执行期间，内存区被分为几部分存储进程执行期间所需的资源（如文本、数据等）。对于不同段的内存区，每一段都有其自己的访问权限集，这些权限由操作系统执行。

（2）操作系统规定一个进程不允许访问其他进程的地址空间，除非这两个进程明确要求共享某一部分彼此的地址空间。

（3）为了提高系统并发运行的能力，操作系统采用了虚拟内存的管理技术。虚拟内存技术是通过进程之间共享内存，从而达到了形式上的内存容量的扩展。虚拟内存的核心思想是：将目前空闲的内存块存到硬盘，然后将空余出的内存供其他进程使用。这样的话，对于用户或是运行于操作系统之上的应用程序而言，只会得到仍有内存可用的信息。

（4）操作系统还提供了其他的高级保护，如对内存交换区进行封装和监视等。

5. 最小特权管理

最小特权原则是在完成某项操作时，系统所赋予用户或进程最小（必不可少）的特权，以保证发生安全事故所造成的损失最小。

最小特权原则的存在是很有必要的。在操作系统中都存在所谓的“超级用户”，UNIX、

Linux操作系统的root用户，Windows操作系统的administrator用户。这些用户拥有最高特权，可以对操作系统所有资源进行配置和管理，这种特权模式非常不利于系统的安全性，恶意用户一旦拿到这些“超级权利”，就会对系统造成极大的损失。基于此，部分安全要求高的系统，采用最小特权管理原则机制，系统中不设置“超级用户”，而是将其权限细粒度地划分给几个用户。这样一旦“超级用户”的用户名和口令丢失，造成的影响也是有限的。

6. 信息通路保护机制

信息通路保护机制的功能有两个，保护正常的显式信道和检测并处理恶意的隐蔽信道。

正常显式信道的保护是通过可信通路实现的，可信通路是一种可以让用户不经过应用层直接与可信计算基之间通信的机制，一般以安全注意键(Security Attention Key，SAK)为基础实现。SAK是键的一个特殊组合，当操作系统检测到用户在终端上键入的SAK，便会启动可信的会话过程。可信通路主要是应用在用户注册或登录时，以防止攻击者利用木马程序窃取用户的用户名和口令，从而保证用户的账号安全。

隐蔽信道在前文已有介绍，它是系统中不受安全策略控制的、违反安全策略的信息泄露路径。隐蔽信道分为隐蔽存储信道和隐蔽定时信道，不同隐蔽信道存在的必要条件不同。

(1) 隐蔽存储信道存在的必要条件如下。

- 发送进程和接收进程具有访问一个共享资源统一属性的权限。
- 发送进程可以修改该共享资源的属性。
- 接收进程可以检测该共享资源属性的变化。
- 存在某种机制，能够启动发送进程和接收进程之间的通信。

(2) 隐蔽定时信道存在的必要条件如下。

- 发送进程和接收进程具有访问一个共享资源统一属性的权限。
- 发送进程和接收进程具有访问一个参考时钟的权限，如访问实时时钟的权限。
- 发送进程能够调节接收进程检测到共享资源属性变化所需的响应时间。
- 存在某种机制，能够启动发送进程和接收进程之间的通信。

对于隐蔽信道的处理，美国橙皮书建议结合使用消除法、宽带限制法和威慑法。这三种方法也常被单独使用。

7. 安全内核

内核是操作系统实现最底层功能的部分。在一个标准的操作系统设计中，如同步、进程间通信、消息传递和中断处理这类操作都是由内核来完成的。

安全内核负责在整个操作系统中实现安全性机制。它提供了与硬件、操作系统和其他部分的安全接口。

用安全内核将系统的安全性功能分离出来有以下的优点。

(1) 对受保护客体的每一次访问都必须通过安全内核的检测。由于安全内核的存在，操作系统就可以保证每个访问都是安全的。

(2) 从系统和用户空间中分离安全机制可以更容易防止其他系统或用户之间的渗透。

(3) 所有的安全功能都是由各自单独的一组编码实现的，所以当某一功能出现问题时，对代码的追踪就会容易一些。

(4) 某项安全机制的修改和测试变得更易实现。

(5) 安全内核仅仅需要实现安全性，所以它的代码量相对来说会精简很多。

(6) 安全内核很容易被分析，可以用形式化方法来验证其是否实现了所有的安全需求。

4.4 操作系统安全模型

4.4.1 安全模型的概念

安全模型是对安全策略所表达的安全需求的简单、抽象和无歧义的描述，它为安全策略和安全策略实现机制的关联提供了一种框架。安全模型描述了对某个安全策略需要用哪种机制来满足；而模型的实现则描述了如何把特定的机制应用于系统中，从而实现某一特定安全策略所需的安全保护。

J. P. Anderson 指出要开发安全系统首先必须建立系统的安全模型。安全模型给出了安全系统的形式化定义，并且正确地综合系统的各类因素。这些因素包括系统的使用方式、使用环境类型、授权的定义、共享的客体（系统资源）、共享的类型和受控共享思想等。构成安全系统的形式化抽象描述，使得系统可以被证明是完整的、反映真实环境的、逻辑上能够实现程序的受控执行的。

安全模型有以下几个特点。

（1）它是精确的、无歧义的。

（2）它是简易和抽象的，容易理解。

（3）它是一般性的，只涉及安全性质，而不过度地牵扯系统的功能或其实现。

（4）它是安全策略的明显表现。

安全模型一般分为两种：形式化的安全模型和非形式化的安全模型。非形式化安全模型仅模拟系统的安全功能；形式化安全模型则使用数学模型，精确地描述安全性及其在系统中使用的情况。

对于高安全级别的操作系统，尤其是对那些以安全内核为基础的操作系统，需要用形式化的开发路径来实现，如图 4.20 所示。这时安全模型就要求是运用形式化的数学符号来精确表达。形式化的安全模型是设计开发高级别安全系统的前提。

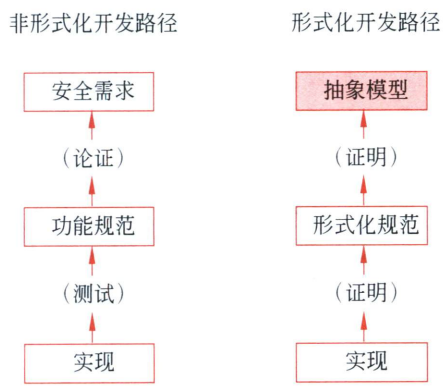

图 4.20 形式化与非形式化安全模型操作系统的开发路径

如果是用非形式化的开发路径，修改一个现有的操作系统以改进它的安全性能，也只能达到中等的安全级别。即使如此，编写一个用自然语言描述的非形式化安全模型也是很值得的，因为安全模型可以保证当设计和安全模型一致时，实现的系统是安全的。

为满足简易性，模型只需要模拟系统中与安全相关的功能，同时可以省略掉系统中的其他与安全无关的功能，这也是系统安全模型和形式化功能规范之间的差别，因为相比较而言形式化功能规范包括了过多的与安全策略无关的系统功能特征。

4.4.2 常见的安全模型介绍

本节主要介绍具有代表性的 BLP 模型、Biba 模型和 Clark-Wilson 完整性安全模型，并在

最后对 Android 安全模型进行分析。

1. BLP 模型

Bell-Lapadula 模型(简称 BLP 模型)是 D. Elliott Bell 和 Leonard J. Lapadula 于 1973 年提出的一种适用于军事安全策略的计算机操作系统安全模型,是典型的信息保密性多级安全模型,它是最早、也是最常用的一种计算机多级安全模型。

BLP 模型通常是多级信息安全模型的设计基础,在 BLP 模型中将主体定义为能够发起行为的实体,如进程;将客体定义为被动的主体行为承担者,如数据,文件等;将主体对客体的访问分为 R(只读)、W(读写)、A(只写)、E(执行),以及 C(控制)等访问模式,其中 C(控制)是该主体用来授予或撤销另一主体对某一客体的访问权限的能力。

BLP 模型建立的访问控制原则是要保证信息流向保密级别更高的主体,即"无向上读,无向下写",这样可以有效地防止低级的用户或进程访问安全级别比他们高的信息资源,其模型如图 4.21 所示。

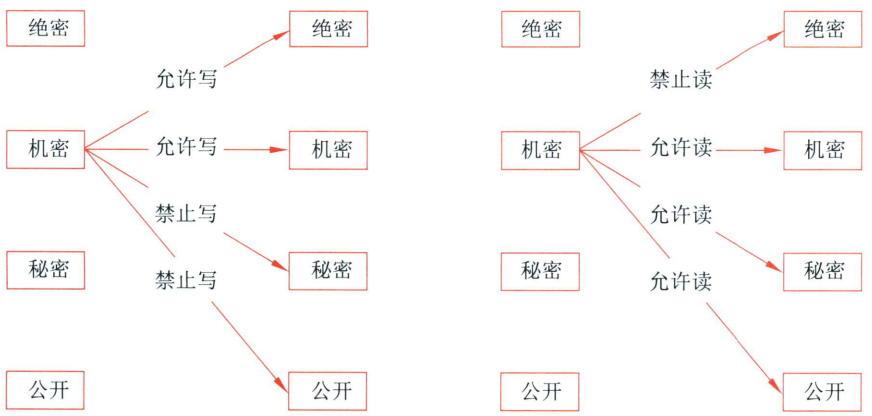

图 4.21 BLP 模型示意图

BLP 模型的安全策略包括两部分:自主安全策略和强制安全策略。自主安全策略允许用户决定主体对客体的访问权限。强制安全策略则执行"无向上读,无向下写",仅允许安全级别高的主体对同级别或安全级别低的客体进行"读",安全级别低的主体仅允许向同级别或较高级别客体进行"写"。BLP 模型用偏序关系可以表示如下:

- Rd,当且仅当 SC(S)≥SC(O),允许读操作;
- Wd,当且仅当 SC(S)≤SC(O),允许写操作。

BLP 模型的局限性如下。

(1) 在 BLP 模型中,可信主体访问权限太大,不符合最小特权原则,应对可信主体的操作权限和应用范围进一步细化。

(2) BLP 模型主要注重保密性控制,控制信息从低安全级传向高安全级,缺少完整性控制,不能控制"向上写(write up)"操作。比如,安全级别低的进程向安全级别高的进程写入数据,安全级别高的进程无法回应低安全级别进程写操作是否完成,或是通过恶意写入数据,覆盖安全级别高的文件原数据。

(3) 操作系统对内存存取管理的要求是必须能够对任何级别进程可读可写,这明显违背了 BLP 模型的"无向上读,无向下写"原则。

(4) 用户的安全级或是信息的安全级可能会发生改变,BLP 模型很难适应。

（5）BLP模型不能解决隐蔽信道问题。

2. Biba模型

在BLP模型的局限性分析中提到，BLP模型注重保密性控制，忽略了完整性这一安全指标。Biba模型是在BLP模型基础上设计的第一个完整性安全模型，其主要应用类似BLP模型的规则来保护信息的完整性。Biba模型模仿BLP模型的信息保密级别，定义了完整的信息完整性级别，模型中主体和客体的概念与BLP模型相同，对系统中的每个主体和每个客体均分配一个级别，称为信息完整性级别，每个信息完整性级别均由两部分组成：密级和范畴。其中，密级是如下分层元素集合中的一个元素：{极重要（Crucial）（C），非常重要（Very Important）（VI），重要（Important）（I）}。此集合是全序的，即C>VI>I。在信息流向方面，Biba模型不允许信息从安全级别低的进程到安全级别高的进程，也就是说用户只能向比自己安全级别低的客体写入信息，以防止非法越权、篡改等行为的产生。

与BLP模型“无向上读，无向下写”不同，Biba模型的原则是“无向上写，无向下读”，如图4.22所示。

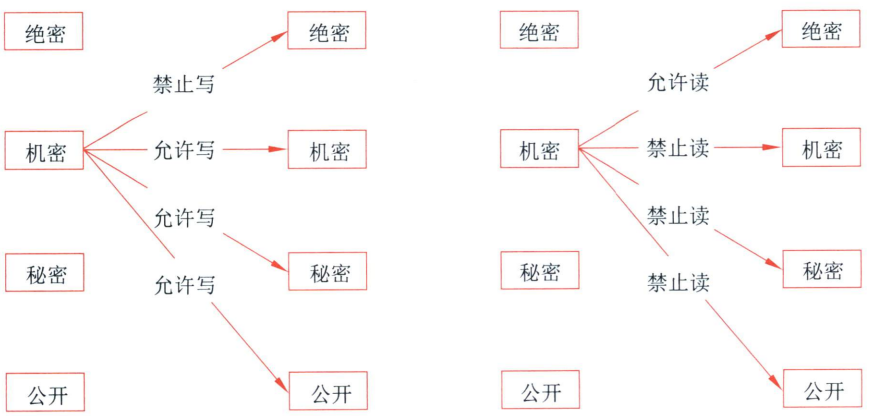

图4.22 Biba模型示意图

这样设计的目的是，保证了完整性高的文件一定是由完整性高的进程产生，低级别的进程无法通过写入数据的形式对高级别的文件进行数据覆盖。Biba模型用偏序关系可以表示如下：

- Rd，当且仅当SC(S)≤SC(O)，允许读操作；
- Wd，当且仅当SC(S)≥SC(O)，允许写操作。

Biba模型的局限性如下。

（1）Biba模型定义的“无向上写，无向下读”规则与BLP模型对立，Biba模型改善了BLP模型所忽略的信息完整性，但是这也在一定程度上丧失了BLP模型保密性强的优点。

（2）Biba模型仅在Multics和VAX等少数几个系统中实现，难以满足实际应用中真正的需求。

3. Clark-Wilson完整性模型

1987年，Clark-Wilson数据完整性安全模型被首次提出。Clark-Wilson是一个评估商用系统安全性的框架，是用以保证商务数据完整性的一个应用级模型，其用途主要是防止对数据进行非法操作，因为其模型并不关注信息的机密性，较容易发生信息的泄露。

Clark-Wilson模型控制数据完整性的方法主要有以下两个。

（1）职责分离原则：规定任意任务从其开始到结束都必须由至少两人参与，其中一人负

责执行任务，一人负责监督证明其完整性。这一原则是为了防止个人可能存在的欺骗行为。

(2) 良构事务原则：规定用户必须用能够确保数据完整性的受控方式来操作数据。这一原则主要是为了防止用户任意操作数据。

Clark-Wilson 模型主要用于授权用户不会在商业应用内对数据进行未经授权的修改、欺骗来保护信息的完整性，在该模型中，用户不能直接访问和操纵客体，而是必须通过一个代理程序来访问客体，从而保护了客体的完整性。使用职责分割来避免授权用户对客体执行未经授权的修改，再次保护数据的完整性。在这个模型中，还需要使用审计功能来跟踪外部进入系统的信息。Clark-Wilson 模型从以下两方面保证数据的完整性。

(1) 防止未经授权的用户进行修改。

(2) 防止授权用户进行未授权或不正确的修改。

所以，Clark-Wilson 模型可以维护内部和外部的统一性。而 Biba 模型只能够做到第一个方面。

4. Android 安全模型

Android 安全模型是基于 Linux 操作系统内核安全性设计的。Android 安全模型的设计特点如下。

(1) 采用多层架构，既保证了用户信息的安全性，也保证了应用程序的灵活性。

(2) 允许开发者充分利用安全架构的灵活性，同时为开发者提供了可靠的默认安全性设置。

(3) 除了考虑恶意软件的威胁，也考虑到了第三方应用程序的恶意攻击。

(4) 在安全保护的同时注重风险控制，受到攻击行为时尽量降低系统的损失。

Android 安全模型主要提供的安全机制如下。

(1) 进程沙箱隔离机制：Android 系统为每个新安装的应用程序分配一个唯一的用户标识(User ID，UID)和一个群组 ID(Group ID，GID)；应用程序及其运行的 Dalvik 虚拟机运行于独立的 Linux 进程空间，根据 UID 的不同保证应用程序都在单独的进程中运行。

(2) 应用程序签名机制：当一个应用程序包(.apk 文件)第一次进行安装时，Android 会检查.apk 文件，该文件必须拥有一个能够识别的开发者数字签名；同一开发者可指定不同的应用程序共享 UID，进而运行于同一进程空间，共享资源。

(3) 权限声明机制：在.apk 文件得到验证后，Android 会查验开发者创建的应用程序对系统进行访问所需要的权限。不同的级别要求应用程序行使此权限时的认证方式不同：normal 级申请即可用；dangerous 级需在安装时由用户确认才可用；signature 与 signatureorsystem 级则必须是系统用户才可用。

(4) 访问控制机制：传统的 Linux 访问控制机制可以确保系统文件与用户数据不受非法访问。

(5) 进程通信机制：为了满足系统进程通信和安全性的要求，Android 引入 Binder 机制。Binder 能够提供类似 COM 与 CORBA 的轻量级远程进程调用(RPC)功能。Binder 基于 Client-Server 通信模式，数据对象仅需复制一次就可以自动传输进程的 UID 信息。相较于传统 Linux 的进程通信机制，Android 的进程通信机制可以实现更加高效、安全的进程通信。

(6) 内存管理机制：Android 安全模型采用低内存清理(LMK)机制，将进程按重要性进行分级、分组。当内存不足时，系统自动清理最低级别进程所占用的内存空间。同时，引入了共享内存机制 Sshmem，使 Android 系统具备清理不再使用共享内存区域的能力。

4.5 攻击和安全技术演示

4.5.1 冰河木马远程控制实验

“冰河木马”主要用于远程监控，能够自动跟踪目标机屏幕变化，同时可以完全模拟键盘及鼠标输入；记录各种口令信息，包括开机口令、屏保口令、各种共享资源口令及绝大多数在对话框中出现过的口令信息；能够限制系统功能，包括远程关机、远程重启计算机、锁定鼠标、锁定系统热键及锁定注册表等多项功能限制；并实现远程的文件操作，包括创建、上传、下载、复制、删除文件或目录、文件压缩、快速浏览文本文件、远程打开文件等多项文件操作功能，是一款强大的远程控制软件。

实验目的：为了理解使用木马进行网络攻击的原理，在虚拟机的 Windows Server 2003 和 Windows XP 环境下操作冰河木马以实现远程控制。

(1) 冰河木马共有两个应用程序，G_Server.exe 是服务器端程序，属于木马受控端程序，G_Client.exe 是木马的客户端程序，属于木马的主控端程序。将下载好的两个程序存放到“实验工具”文件夹中，右击“实验工具”文件夹，选择“共享和安全”选项，选择“如果您知道在安全方面的风险，但又不想运行向导就共享文件，请单击此处”选项，选择“只启用文件共享”选项，选中“在网络上共享这个文件夹”复选框，如图 4.23 所示。

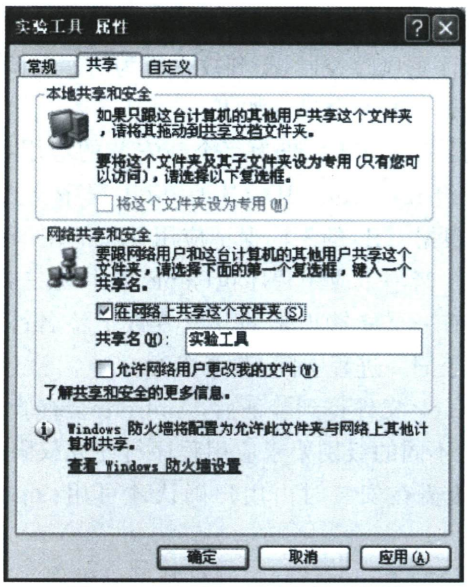

图 4.23 “实验工具”属性配置图

(2) 打开“目标机”的控制台，打开“我的电脑”，在地址栏输入“\\192.168.20.16”，如果无法连接，两台主机先 ping 一下。在种木马之前，在受控端计算机中 cmd 运行 regedit 打开注册表，查看打开 txtfile 的应用程序注册项“HKEY_CLASSES_ROOT\txtfile\shell\open\command”，可以看到打开.txt 文件默认值是“%SystemRoot%\system32\NOTEPAD.EXE %1”，再打开受控端计算机的 C:\windows\system32 文件夹，此时找不到 sysexplr.exe 文件。

(3) 在受控端计算机中配置服务端的 G_Server.exe 程序，选择“设置”→“配置服务器程序”菜单选项对服务器进行配置，如图 4.24 所示。

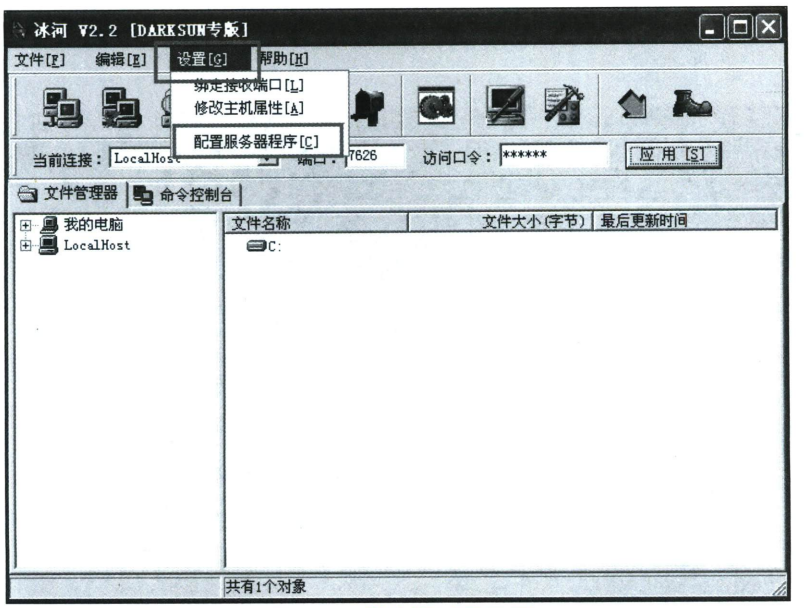

图 4.24 服务器端配置图

(4) 在服务器配置对话框中对待配置文件进行设置，找到服务器程序文件 C:\Documents and Settings\Administrator\桌面\实验工具\冰河木马\G_Server.exe，打开该文件；再在访问口令框中输入 123456，然后单击“确定”按钮，就对服务器配置完毕。此时打开受控端计算机的注册表，查看.txt 文件的应用程序注册项“HKEY_CLASSES_ROOT\txtfile\shell\open\command”，可以发现，这时它的值为“C:\WINDOWS\system32\SYSEXPLR.EXE%1”；再打开受控端计算机的 C:\windows\system32 文件夹，这时可以找到 sysexplr.exe 文件。

(5) 在主控端计算机中，双击 G_Client.exe 图标，打开木马的客户端程序(主控程序)，在该界面的访问口令编辑框中输入访问口令：123456，设置访问口令，然后单击“应用”按钮，具体如图 4.25 所示。

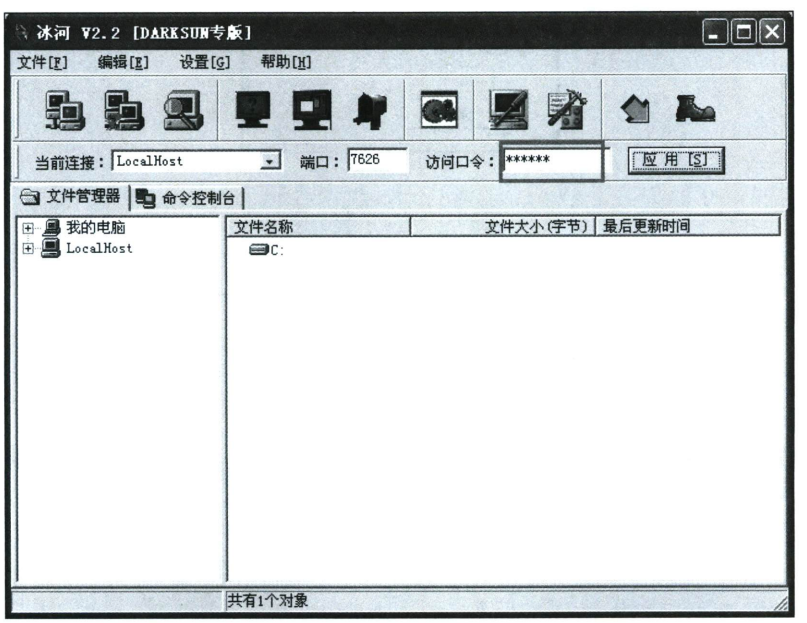

图 4.25 访问口令设置图

（6）在主控端程序中添加需要控制的受控端计算机之前，先在受控端计算机中查看其IP地址，具体如图4.26所示。

图4.26 IP地址查询结果图

（7）之后在主控端计算机程序中添加受控端计算机，具体如图4.27所示。

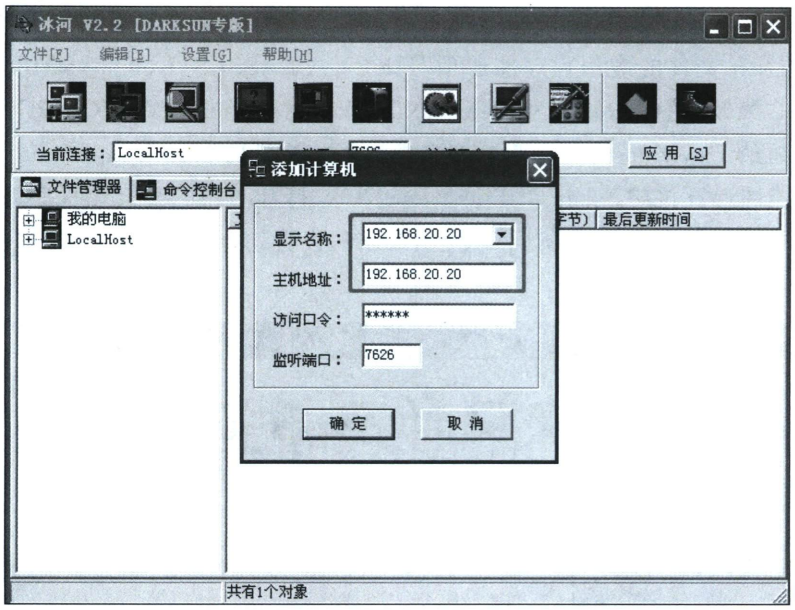

图4.27 添加受控端计算机示例图

（8）将受控端计算机添加后，即可以浏览受控端计算机中的文件系统。进行上传文件测试，上传到C:\Documents and Settings文件夹，具体如图4.28和图4.29所示。

（9）文件上传成功后，登录至受控端计算机进行验证，发现完全吻合，具体如图4.30所示。

4.5.2 Windows操作系统的安全配置

操作系统的安全配置是整个操作系统安全策略的核心，其目的就是从系统根源构筑安全

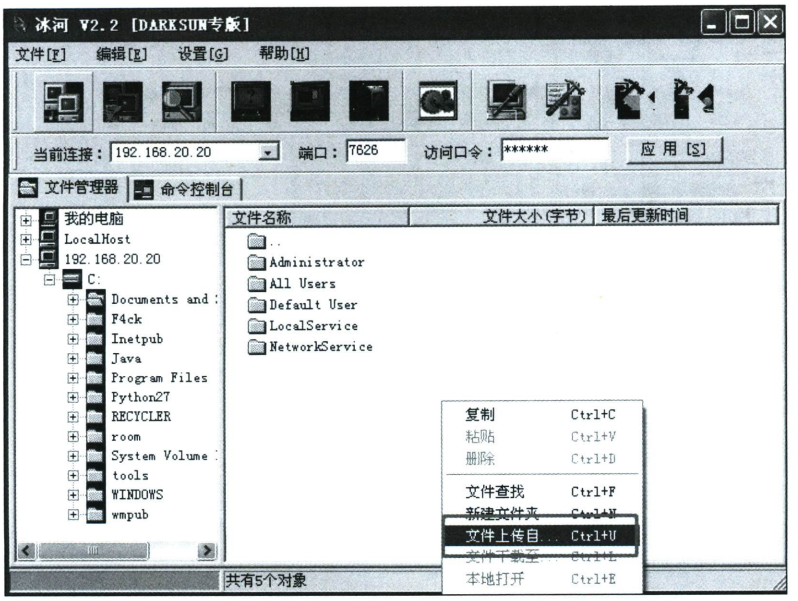

图 4.28　文件上传示例图 1

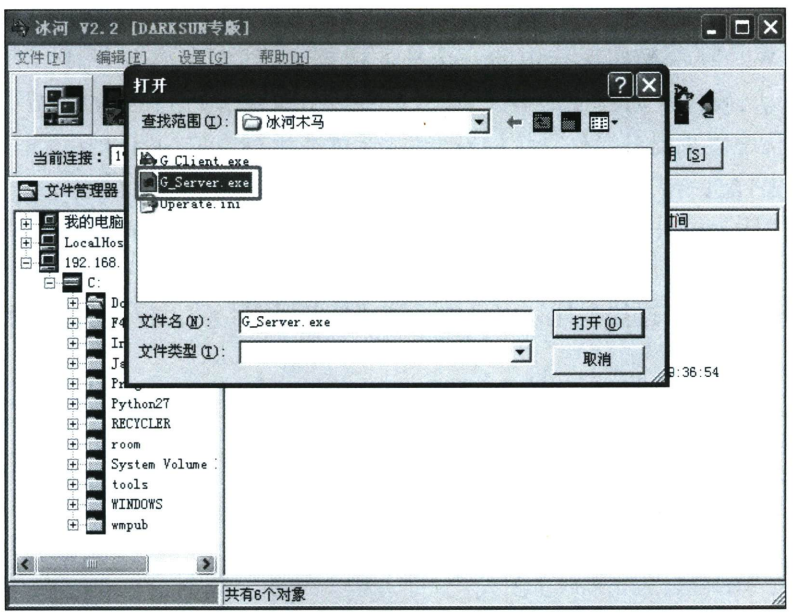

图 4.29　文件上传示例图 2

防护体系，通过用户和密码管理、共享设置、端口管理和过滤、系统服务管理、本地安全策略、外部工具使用等手段形成一整套有效的系统安全策略，在保证系统使用功能的基础上提高其安全性。

通过安全策略的配置，可以控制访问计算机的用户、授权用户使用计算机上的哪些资源、确定用于管理域账户或本地用户账户的密码策略、在事件日志中记录并查看用户或组的操作等。本节以 Windows 10 操作系统为例，对 Windows 操作系统进行安全配置。

实验目的：掌握操作系统安全相关的基础知识，熟悉 Windows 操作系统的安全配置。

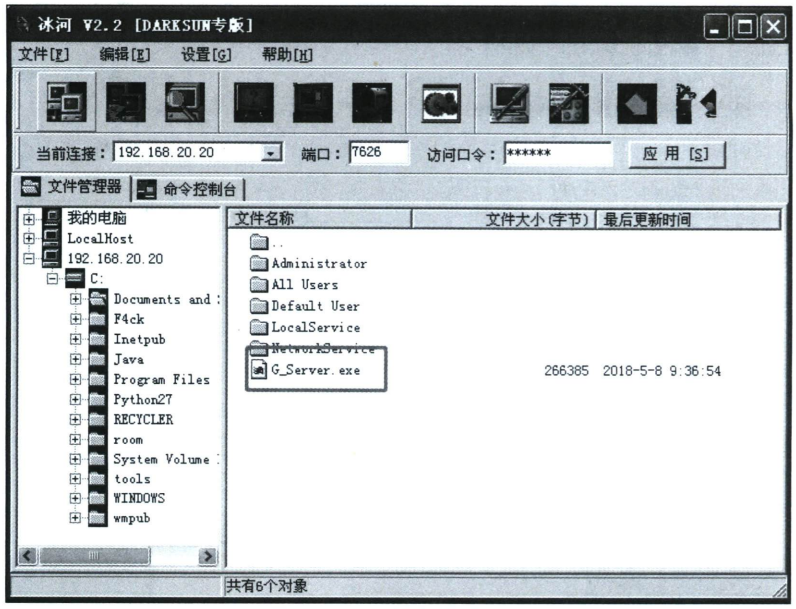

图 4.30 文件上传结果图

1. Windows 系统注册表配置

注册表是 Windows 操作系统中的一个核心数据库，其中存放着各种参数，直接控制着 Windows 的启动、硬件驱动程序的装载，以及一些 Windows 应用程序的运行。因此，可以修改注册表的默认配置来提升操作系统的安全性，通过在运行窗口的打开文本框中输入 regedit 命令打开注册表，配置 Windows 系统注册表中的安全项。

（1）找到注册表中“HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Service”下的 RemoteRegistry 选项，如图 4.31 所示，右击选择“删除”选项以彻底关闭 Windows 远程注册表服务，避免攻击者连接到计算机操作系统的任意服务，并且避免攻击者通过命令行指令将服务开启。

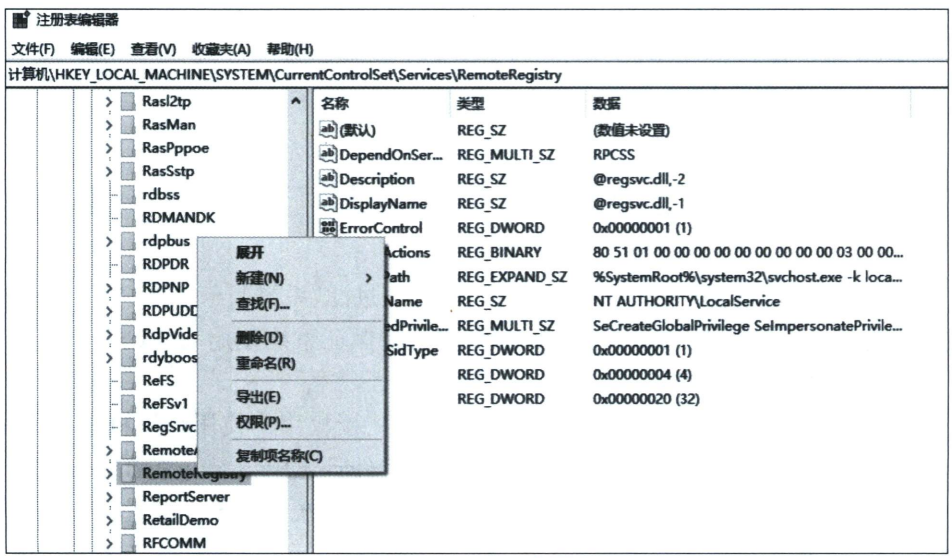

图 4.31 关闭远程注册表服务示例图

（2）查找注册表中“HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\Lsa”的 restrictanonymous 选项，右击选择“修改”选项，如图 4.32 所示，将 restrictanonymous 选项设置为 1 以防范 IPC$（internet process connection）攻击，避免攻击者获取 IPC$ 的网络文件共享能力。

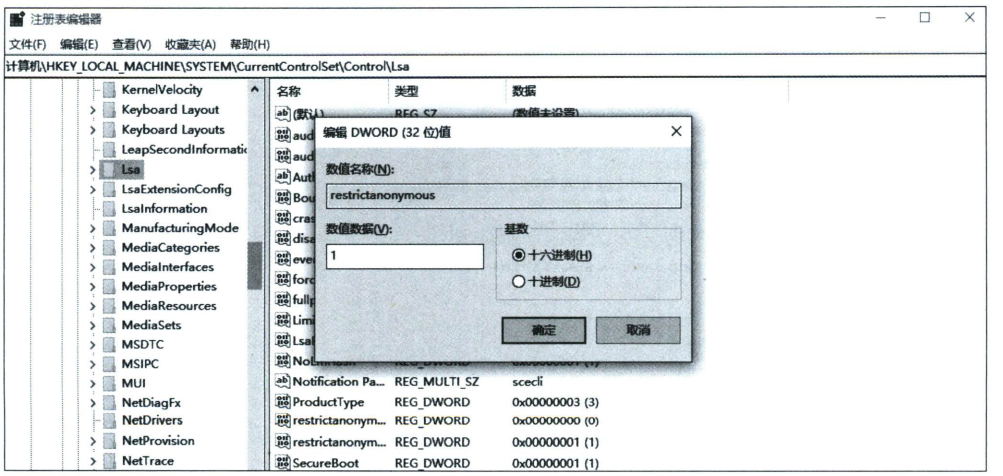

图 4.32　修改资源共享权限示例图

（3）由于在默认安装 Windows 系统的情况下，所有的硬盘都是隐藏共享的，其他用户可以在网络中访问本地的资源，虽然对其访问需要超级用户的密码，但这仍然是一个潜在的安全隐患。在注册表中找到“HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Services\LanmanServer\Parameters”选项，在该选项的右边空白处右击选择新建 DWORD 值，添加键值 AutoShareServer（类型为“REG_DWORD”，值为 0），如图 4.33 所示，即可关闭默认共享。

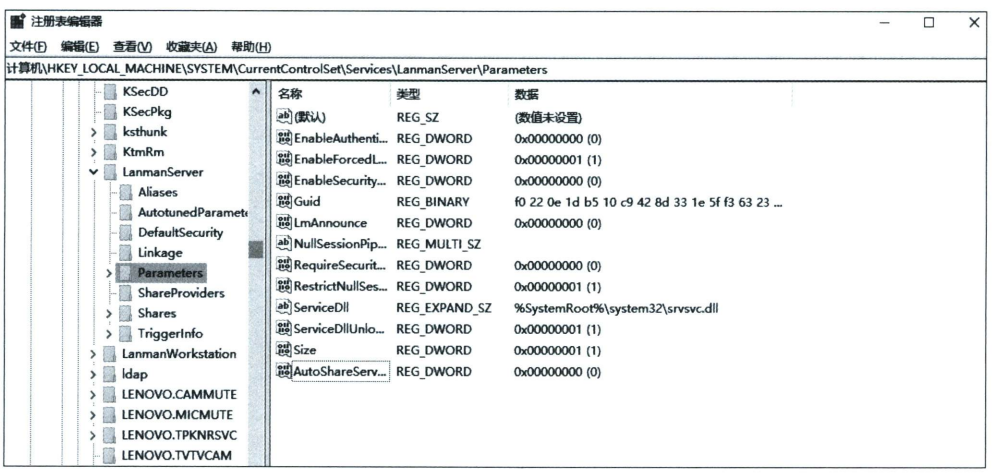

图 4.33　关闭默认共享示例图

2. Windows 本地安全策略配置

本地安全策略是对登录到计算机上的账号定义的一些安全设置，在没有活动目录集中管理的情况下，本地管理员必须为计算机进行设置以确保其安全。例如，限制用户如何设置密码、通过账户策略设置账户安全性、通过锁定账户策略避免他人登录计算机、指派用户权限等。通过“控制面板\管理工具\本地安全策略”可以配置本地的安全策略（命令为 gpedit.msc 和 secpol.msc）。

（1）默认安装 Windows 系统的情况下，Windows 系统允许匿名用户执行某些活动。在“本地安全策略”左侧列表的“安全设置”目录树中逐层展开“本地策略”“安全选项”选项，查看右侧的相关策略列表，启用“网络访问：不允许 SAM 账户和共享的匿名枚举”，如图 4.34 所示，阻止对不需要维护相互信任关系的信任域中的用户访问的授权。

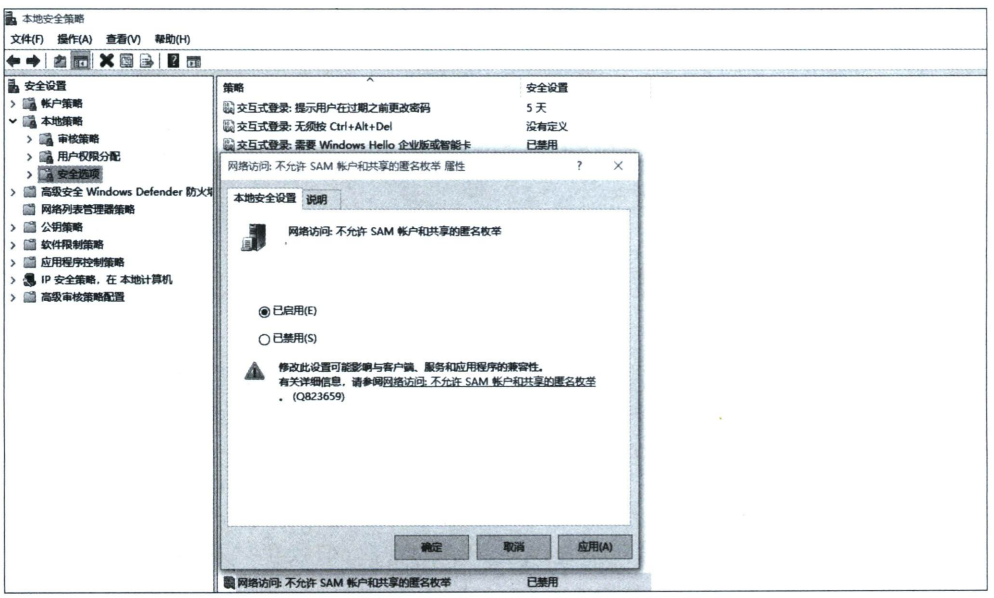

图 4.34 SAM 账户和共享匿名枚举禁用图

（2）在“安全选项”中设置不显示上次登录的用户名，如图 4.35 所示。

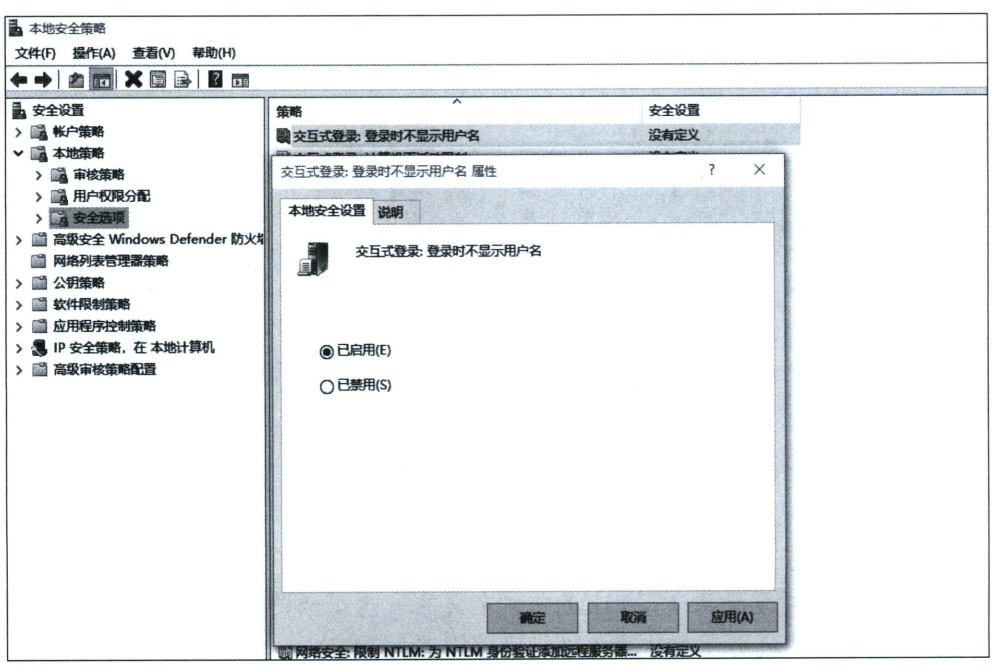

图 4.35 用户名显示禁用图

（3）在“本地安全策略”左侧列表的“安全设置”目录树中逐层展开“账户策略”“密码策略”，如图 4.36 所示，双击“密码必须符合复杂性要求”，启用该功能，以便在设置密码时进行密

码复杂性检查；设置密码长度最小值，提升密码复杂度，将密码长度设置在 6 位以上；设置密码最长存留期，通过提高修改密码的频率来提升安全系数，双击“密码最长使用期限”选项将密码作废期设置为 60 天，则用户每次设置的密码只在 60 天内有效。

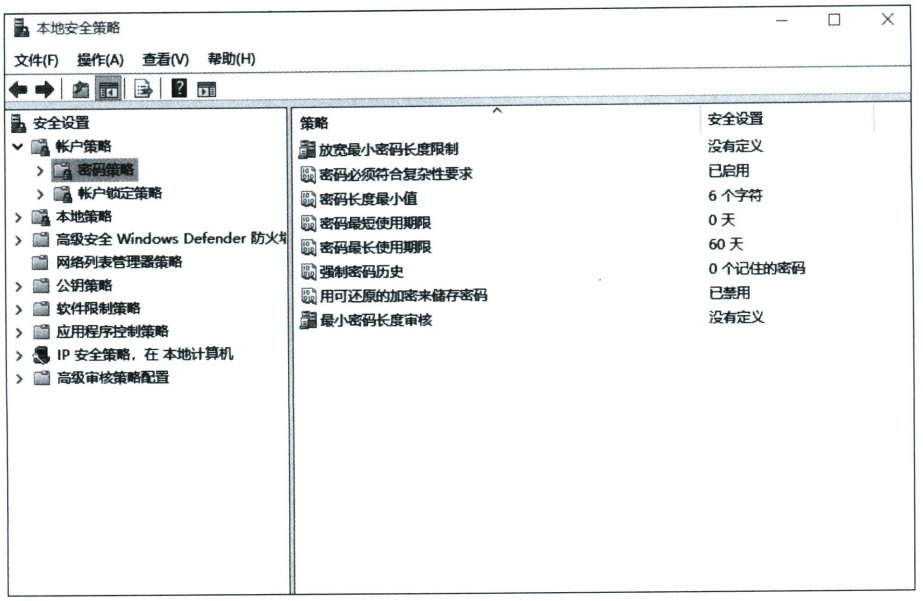

图 4.36　密码策略设置图

3. Internet 安全设置

IE 浏览器是普通网民使用较频繁的软件之一，也是常常受到网络攻击的软件，人们往往都会装不少的三方软件来避免这种攻击。其实，在 IE 浏览器中有不少容易被人们忽视的安全设置，通过这些设置能够在很大程度上避免网络攻击。打开 IE 浏览器，选择“工具”→“Internet 选项”→“安全”选项进行 IE 浏览器的安全设置。

(1) 在 Internet 选项中自定义安全级别，如图 4.37 所示，选中需要设置安全级别的区域后进行调节，或单击“自定义级别”按钮自行设置。

(2) 设置添加受信任和受限制的站点，如图 4.38 所示。

4. EFS 加密配置

加密文件系统(encrypting file system，EFS)是 Windows 系统特有的一个实用功能，对于 NTFS 卷上的文件和数据，都可以直接被操作系统加密保存，在很大程度上提高了数据的安全性。在计算机里面选择要进行加密的文件，然后右击选择“属性”选项，在“属性”对话框里面选择“高级”选项，在“高级属性”对话框里面勾选“加密内容以便保护数据”复选框，如图 4.39 所示，保证其他非授权用户无法访问经过 EFS 加密的数据。

5. 日志的查询

Windows 日志是记录系统中硬件、软件和系统问题的信息，同时还可以监视系统中发生的事件，用户可以通过它来检查错误发生的原因，或者寻找受到攻击时攻击者留下的痕迹。Windows 日志包括系统日志、应用程序日志和安全日志。查看这三种日志需要以管理员身份登录系统，选择“控制面板”→“管理工具”→“事件查看器”→“Windows 日志”选项，双击“应用程序”就可以看到系统记录的应用程序日志，如图 4.40 所示，在右侧的详细信息窗格中双击某一条信息，就能够看到该信息所记录事件的详细信息。安全日志和系统日志可用同样的方法查看。

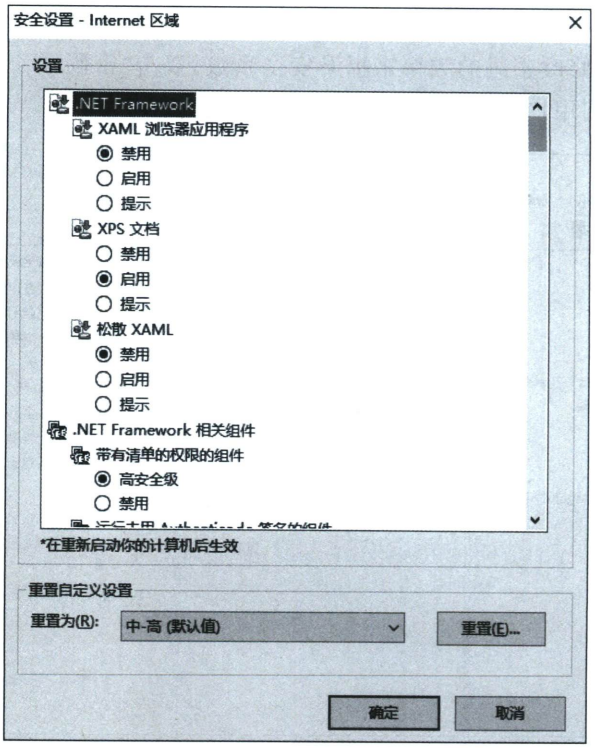

图 4.37　安全级别设置图

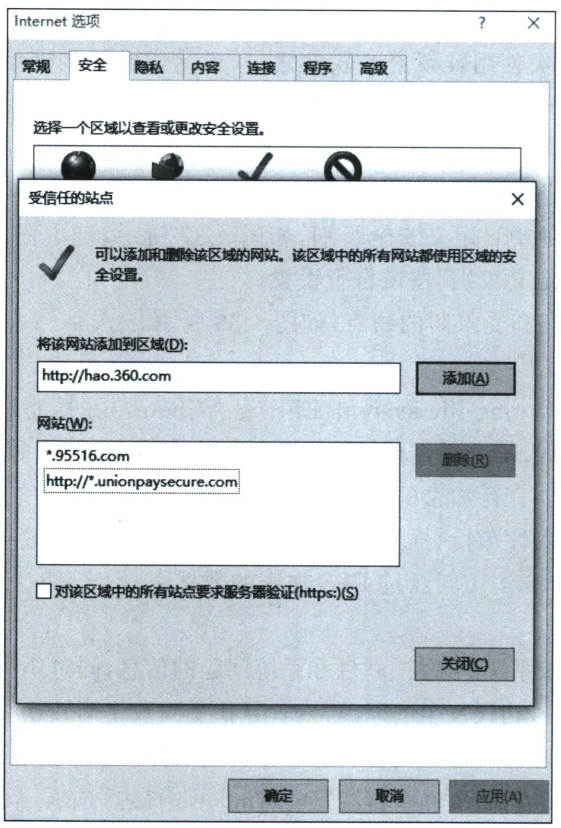

图 4.38　受信任的站点配置图

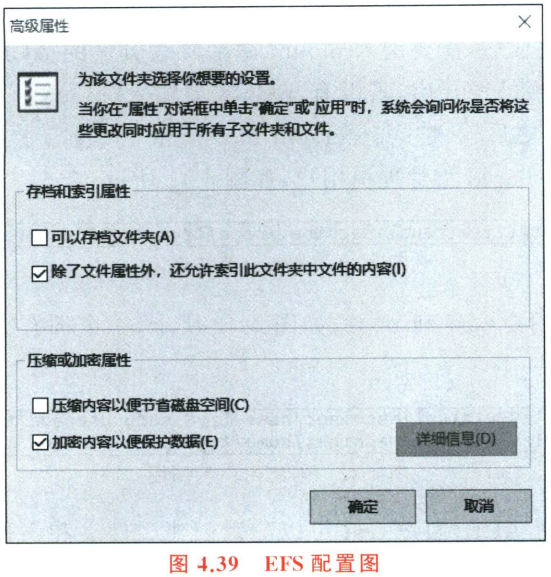

图 4.39　EFS 配置图

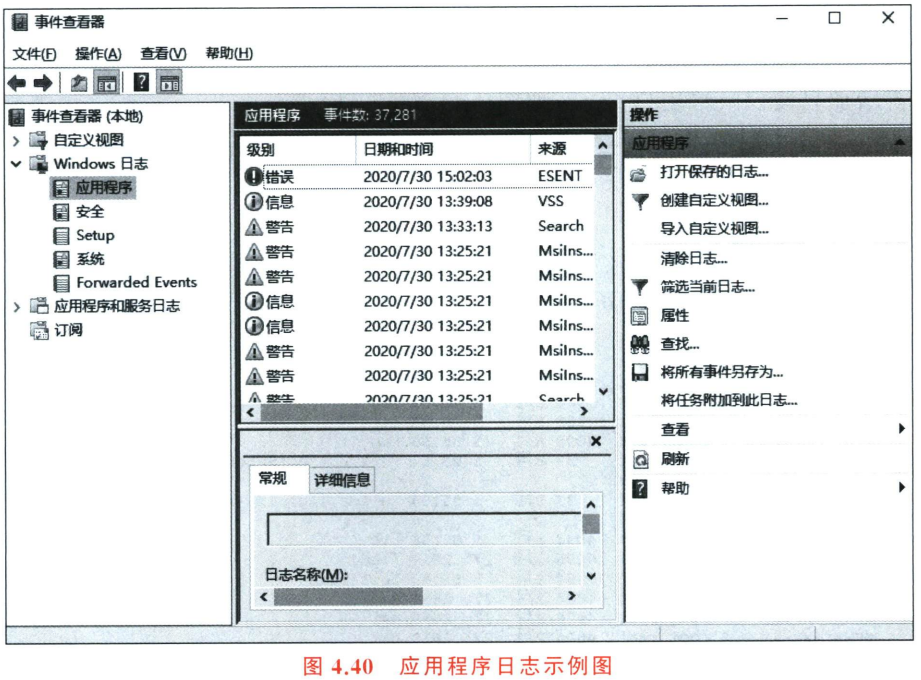

图 4.40　应用程序日志示例图

4.5.3　Linux 访问控制列表

Linux 系统有一种被称为访问控制列表(ACL)的权限控制方法，它是一种权限分配之外的普遍范式，默认情况下需要确认 3 个权限组：owner、group 和 other，使用 ACL 可以增加权限给其他用户或组别，灵活挑选设置复杂的配置和权限以满足组织的不同需求。

本节以虚拟机环境下的 Linux 操作系统 Ubuntu 16.04.6 为例，进行 Linux 访问控制列表配置实验。

实验目的：掌握访问控制列表的基本知识，熟悉 Linux 操作系统访问控制列表的配置。

（1）安装工具来管理 ACL。在 Ubuntu 系统的命令行窗口输入下列命令“sudo apt-get install acl”，安装 ACL 完成后，需要键入 mount 激活磁盘分区的 ACL 功能，这样才可以使用它，再检查 ACL 功能是否已经开启，若没有开启，需要输入“mount -o remount，acl /”（[/]分区）临时开启 ACL 权限，之后输入“sudo mount / -o remount”重新挂载分区。

（2）创建一个目录/shared 给管理员用户，如图 4.41 所示。

```
root@lzy-virtual-machine:/home/lzy# mkdir /shared
```

图 4.41 创建目录示例图

（3）之后新建两个用户 test 和 test2，如图 4.42 所示，一个拥有完整权限，另一个只有读权限。

```
root@lzy-virtual-machine:/home/lzy# sudo useradd test
root@lzy-virtual-machine:/home/lzy# sudo useradd test2
```

图 4.42 创建用户示例图

（4）为用户 test 设置 ACL，用户 test 可以随意创建文件夹和文件、访问/shared 目录，如图 4.43 所示。

```
root@lzy-virtual-machine:/home/lzy# sudo setfacl -m u:test:rwx /shared
```

图 4.43 用户 test 权限配置图

（5）再增加只读权限给用户 test2，如图 4.44 所示。

```
root@lzy-virtual-machine:/home/lzy# sudo setfacl -m u:test2:rx /shared
```

图 4.44 用户 test2 权限配置图

（6）读取 ACL，若权限后多一个“+”标记，表明 ACL 已经设置成功，如图 4.45 所示。

```
root@lzy-virtual-machine:/home/lzy# ls -l /
total 104
drwxr-xr-x   2 root root  4096 8月   5 05:58 bin
drwxr-xr-x   3 root root  4096 8月   5 05:58 boot
drwxrwxr-x   2 root root  4096 8月   5 05:52 cdrom
drwxr-xr-x  18 root root  3980 8月   5 17:09 dev
drwxr-xr-x 130 root root 12288 8月   5 20:04 etc
drwxr-xr-x   3 root root  4096 8月   5 05:56 home
lrwxrwxrwx   1 root root    33 8月   5 05:57 initrd.img -> boot/initrd.img-4.15
.0-45-generic
lrwxrwxrwx   1 root root    33 8月   5 05:50 initrd.img.old -> boot/initrd.img-
4.15.0-45-generic
drwxr-xr-x  22 root root  4096 8月   5 05:58 lib
drwxr-xr-x   2 root root  4096 2月  27  2019 lib64
drwx------   2 root root 16384 8月   5 05:49 lost+found
drwxr-xr-x   2 root root  4096 2月  27  2019 media
drwxr-xr-x   2 root root  4096 2月  27  2019 mnt
drwxr-xr-x   2 root root  4096 2月  27  2019 opt
dr-xr-xr-x 226 root root     0 8月   5 17:08 proc
drwx------   3 root root  4096 2月  27  2019 root
drwxr-xr-x  25 root root   780 8月   5 17:14 run
drwxr-xr-x   2 root root 12288 8月   5 05:58 sbin
drwxrwxr-x+  2 root root  4096 8月   5 20:04 shared
drwxr-xr-x   2 root root  4096 8月   5 17:04 snap
```

图 4.45 ACL 状态图

（7）若要具体查看/shared 文档的 ACL，则需要在命令行窗口运行“sudo getfacl /shared”，结果如图 4.46 所示。

（8）如果想要移除/shared 文档的 ACL，输入“sudo setfacl -x u：test /shared”，结果如图 4.47 所示。

（9）如果想要立即擦除/shared 文档所有的 ACL 条目，可输入“sudo setfacl -b /shared”，结果如图 4.48 所示。

```
root@lzy-virtual-machine:/home/lzy# sudo getfacl /shared
getfacl: Removing leading '/' from absolute path names
# file: shared
# owner: root
# group: root
user::rwx
user:test:rwx
user:test2:r-x
group::r-x
mask::rwx
other::r-x
```

图 4.46 shared 文档状态图

```
root@lzy-virtual-machine:/home/lzy# sudo setfacl -x u:test /shared
root@lzy-virtual-machine:/home/lzy# sudo getfacl /shared
getfacl: Removing leading '/' from absolute path names
# file: shared
# owner: root
# group: root
user::rwx
user:test2:r-x
group::r-x
mask::r-x
other::r-x
```

图 4.47 用户 test 权限移除结果图

```
root@lzy-virtual-machine:/home/lzy# sudo setfacl -b /shared
root@lzy-virtual-machine:/home/lzy# sudo getfacl /shared
getfacl: Removing leading '/' from absolute path names
# file: shared
# owner: root
# group: root
user::rwx
group::r-x
other::r-x
```

图 4.48 所有用户权限移除结果图

4.6 本章小结

本章首先对操作系统的概念和其面临的安全隐患进行了简单概述；其次，描述了操作系统内核安全、访问控制和主要的安全机制；再次，介绍了安全模型的概念和常见的安全模型，包括 BLP 安全模型、Biba 安全模型、Clark-Wilson 完整性模型和 Android 安全模型；最后，结合本章的内容，对相应的攻击和安全技术进行了教学实验。

思 考 题

1. 你的个人计算机所用操作系统的安全级别是什么？它是安全操作系统吗？
2. 在相关网站上查找 Windows 操作系统最新的安全漏洞，分析其类型和原理。
3. 请试着复述 WananCry 感染用户计算机的过程。
4. 找一套最新版本的 Linux 系统，实际测试一下其所提供的安全功能。
5. 利用最新版本的 Windows 系统，实际测试一下其所提供的安全功能。
6. 请分别简述自主访问控制、强制访问控制和基于角色的访问控制的基本内容，以及它们之间的异同点。
7. 访问控制与用户的标识与鉴别机制之间存在怎样的关系？
8. 请举出 Windows 操作系统提供的安全机制中属于 DAC 和 MAC 的例子。